

Санкт-Петербургский государственный университет
информационных технологий, механики и оптики.

Методическое пособие по проведению лабораторных работ по
дисциплине «Основы элементов автоматики».

Санкт-Петербург
2026 г.

СОДЕРЖАНИЕ:

1. Введение.....	3
2. Описание лабораторного стенда.....	3
3. Описание материнской платы.....	4
4. Использование пакета программ	5
4.1. Работа в интегрированной среде разработки Silicon Laboratories IDE.....	5
4.2. Пример работы с платой и IDE.....	Ошибка! Закладка не определена.
5. Лабораторные работы.....	11
5.1. Работа №1 «Работа со светодиодным матричным дисплеем»	11
5.2. Лабораторная работа №2 «Контроль температуры»	12
5.3. Работы №3,4 «Управление электродвигателями»	13
5.4. Работа №5 «Символьный экран и матричная клавиатура»	17
6. Описание встроенных периферии МК.....	25
6.1. Порты ввода/вывода. Матрица назначения выводов портов.....	25
6.1.1. Приоритетный декодер матрицы.....	Ошибка! Закладка не определена.
6.1.2. Инициализация портов ввода/вывода	Ошибка! Закладка не определена.
6.1.3. Порты ввода/вывода общего назначения	Ошибка! Закладка не определена.
6.2. Таймеры/счетчики.....	27
6.2.1. Таймер 0 и Таймер 1	Ошибка! Закладка не определена.
6.2.2. Таймер 2	Ошибка! Закладка не определена.
6.2.3. Таймер 3	Ошибка! Закладка не определена.
6.3. Модули сравнения фиксации (программируемый массив счетчиков)	
Ошибка! Закладка не определена.	
6.3.1. Таймер/счетчик модуля ПМС	Ошибка! Закладка не определена.
6.3.2. Модули захвата/сравнения.....	Ошибка! Закладка не определена.
6.3.3. Режим сторожевого таймера	Ошибка! Закладка не определена.
6.4. Интерфейс SPI	38
6.4.1. Описание сигналов.....	38
6.4.2. Функционирование SPI0 в ведущем режиме	Ошибка! Закладка не определена.
6.4.3. Функционирование SPI0 в ведомом режиме.....	Ошибка! Закладка не определена.
6.4.4. Источники прерываний модуля SPI0	Ошибка! Закладка не определена.
6.4.5. Тактирование	Ошибка! Закладка не определена.
6.4.6. Регистры специального назначения модуля SPI0....	Ошибка! Закладка не определена.
6.5. Интерфейс UART	44
6.5.1. Усовершенствованный режим генерации скорости передачи данных	
Ошибка! Закладка не определена.	
6.5.2. Режимы работы UART0	Ошибка! Закладка не определена.
6.6. Аналогово-цифровой преобразователь.....	49
6.6.1. Аналоговый мультимплексор.....	Ошибка! Закладка не определена.
6.6.2. Режимы работы АЦП0.....	Ошибка! Закладка не определена.
6.6.3. Программируемый детектор диапазона АЦП0	Ошибка! Закладка не определена.
6.7. Цифро-аналоговый преобразователь	57

- 6.7.1. Обновление выходного сигнала ЦАП0..... **Ошибка! Закладка не определена.**
- 6.7.2. Форматирование входных данных ЦАП **Ошибка! Закладка не определена.**
- 6.8. Генераторы..... **Ошибка! Закладка не определена.**

1. Введение

В данном методическом пособии приводится описание лабораторных стендов и методики выполнения лабораторных работ по курсу «Микропроцессорная техника». Целью данных лабораторных работ является освоение студентами навыков программирования реальных систем на базе микроконтроллеров. Для достижения этой цели используются лабораторные стенды, реализующие типовые функции систем на базе микроконтроллеров. К подобным функциям относятся: реализация символьной и графической индикации и клавиатуры, управление двигателями, обработка сигналов различных датчиков, обеспечение интерфейса с компьютером. Центральным узлом данных лабораторных стендов является микроконтроллер GD32F303CC фирмы GigaDevices, имеющий широко распространенную архитектуру ARM Cortex M4. Для разработки и отладки программного обеспечения для лабораторного стенда используется инструментальная среда разработки Keil uVision, работающая на персональном компьютере под управлением операционной системы Windows.

2. Описание лабораторного стенда

Применяемый лабораторный стенд включает в себя отладочную материнскую плату, набор дополнительных дочерних плат для выполнения лабораторных работ, блок питания платы, программатор/отладчик, компьютер. Отладочная материнская плата соединена с компьютером с помощью программатора/отладчика, подключаемого к USB-порту компьютера. Посредством программатора/отладчика в материнскую плату лабораторного стенда заносится отлаживаемое программное обеспечение, разработанное на компьютере, а также, реализуются различные режимы его отладки. Помимо отладочного, материнская плата имеет отдельный USB-интерфейс, который может быть использован для взаимодействия реализуемых систем с компьютером.

В материнскую плату лабораторного стенда мезонинным способом вставляются различные дочерние платы, каждая из которых предназначена для реализации какой-то конкретной системы. К отдельным дочерним платам подключается дополнительное периферийное оборудование.

3. Описание материнской платы

Структурная схема материнской платы лабораторного стенда представлена на Рис. 3.1.

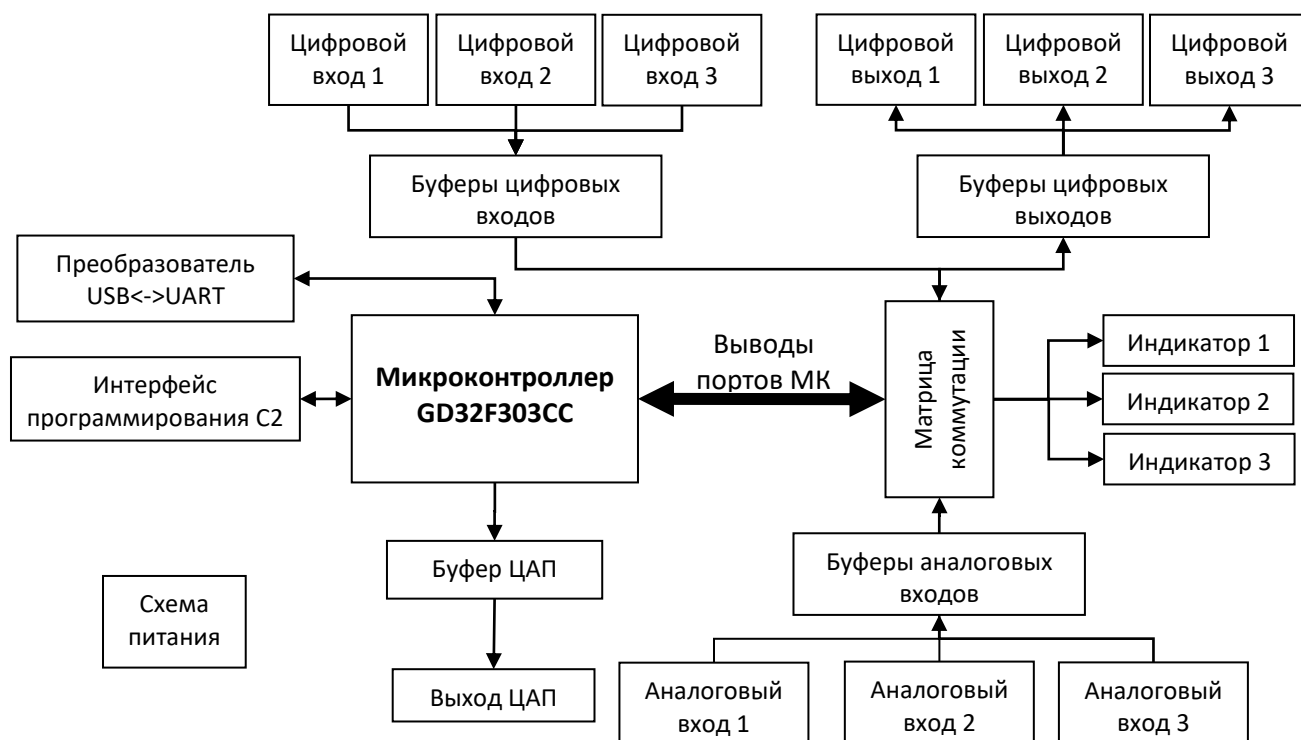


Рис. 3.1 Блок-схема печатной платы на основе МК C8051F361

Здесь:

- *микроконтроллер (МК) GD32F303CC*. МК – микросхема, предназначенная для управления электронными устройствами. Типичный МК сочетает в себе функции процессора и периферийных устройств, может содержать ОЗУ и ПЗУ. По сути, это однокристальный микрокомпьютер, способный выполнять простые задачи. Служит для обработки команд программы, записанной (защитой) в его память. Более подробно данный МК описан в техническом описании [GD32F303xx_Datasheet_Rev1.7.pdf](#) и [GD32F30x_User_Manual_EN_v2.3-2.pdf](#);
- *цифровые входы* служат для ввода логической информации в плату;
- *цифровые выходы* служат для вывода логической информации из платы;
- *буферы цифровых входов* и *буферы цифровых выходов* обеспечивает развязку между сигналами с внешних устройств и портами МК. Это позволяет избежать перегрузки выводов портов МК и ограничить входные сигналы;
- *матрица коммутации* представляет собой два разъёма: один подключен к выводами портов МК, второй – к периферийным устройствам платы (индикаторы, входы/выходы и т.д.). Матрица позволяет подключать выводы портов МК к используемым в данный момент устройствам, что позволяет работать МК как с устройствами данной платы (например портами ввода/вывода), так и с другими платами (например – ЖК-дисплей, плата электропривода), которые вставляются в разъем матрицы;

- *индикаторы* – три светодиода. При подключении выводов МК к индикаторам посредством матрицы коммутации, появляется возможность управлять ими программно, т.е. выводить на них логическую информацию;
- *выход цифро-аналогового преобразователя (ЦАП) (аналоговый выход)* позволяет выводит из платы аналоговый сигнал встроенного в МК цифро-аналогового преобразователя;
- *буфер ЦАП* служит для масштабирования выходного ЦАП;
- *аналоговые входы* служат для ввода аналоговых сигналов в плату для последующей оцифровки на аналого-цифровом преобразователе (АЦП) МК;
- *буфер аналогового входа (буфер АЦП)* - служит для масштабирования входного сигнала аналого-цифрового преобразователя МК;
- *преобразователь USB-UART* обеспечивает обмен данными между материнской платой и компьютером. Связывает USB-порт компьютера с UART-интерфейсом.
- *интерфейс программирования* служит для подключения программатора/отладчика к материнской плате по интерфейсу C2;
- *схема питания* служит для питания платы.

4. Использование пакета программ

Для разработки и отладки программ под данные микроконтроллеры используется программный продукт Keil uVision. Он представляет собой интегрированную среду разработки, которая позволяет писать исходный код на языках Си или Ассемблер, производить его компиляцию и управлять программатором/отладчиком, с помощью которого осуществляется загрузка прошивки в МК и его внутрисхемная отладка.

4.1. Работа в интегрированной среде разработки Keil uVision.

Для запуска Keil uVision можно воспользоваться ярлыком который находится либо на Рабочем столе, либо в меню Пуск->Программы-> Keil uVision5 -> Keil uVision5.

Главное окно инструментальной среды разработки Keil uVision представлено на рис.4.1.

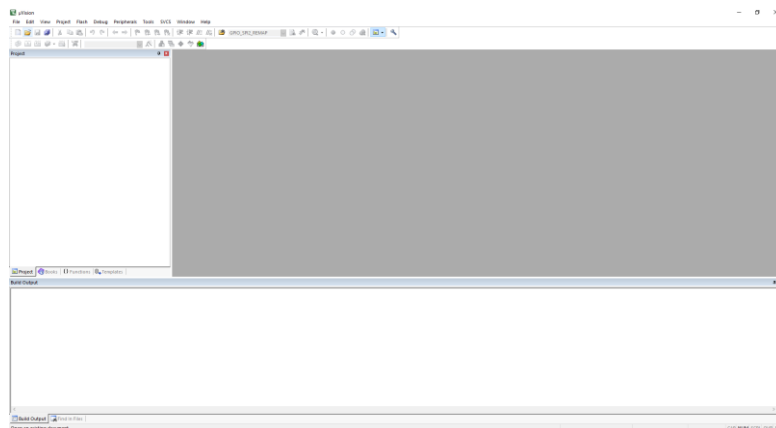


Рис. 4.1 Главное окно Keil uVision IDE

Для начала работы необходимо создать новый проект: Project->New uVision Project. В появившемся окне (рис 4.2) нужно выбрать папку и имя проекта (лучше создать отдельную папку для каждого проекта и в ее имени указать номер учебной группы и номер бригады, чтобы самим было удобно потом находить свой проект). После этого появится окно выбора микроконтроллера (рис 4.2). В нём нужно выбрать семейство микропроцессоров: GD32F303, а в нём микроконтроллер GD32F303CC.

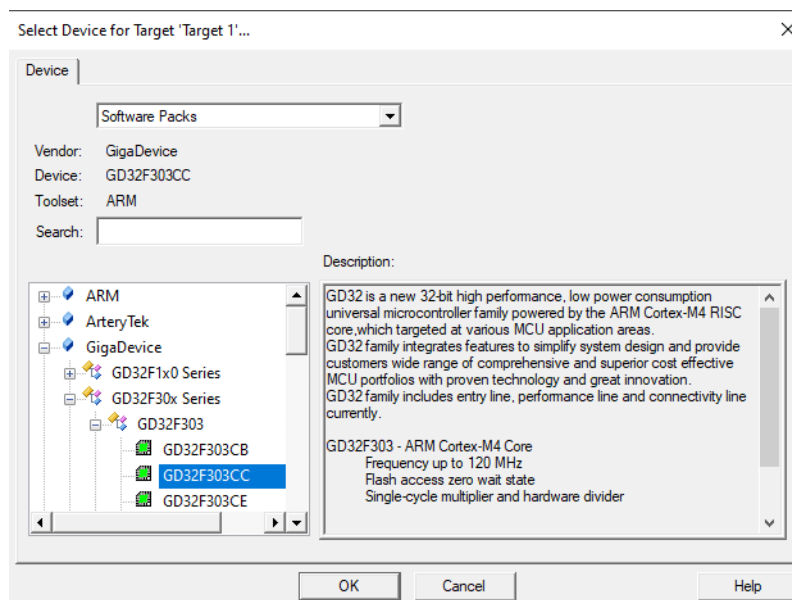


Рис. 4.2 Окно выбора микроконтроллера

Далее откроется окно необходимого программного окружения (рис 4.2), в котором необходимо выбрать драйверы периферийных устройств, которые предполагается использовать в проекте. Если в столбце Sel появляются жёлтые секции, это означает, что выбранные элементы требуют подключения дополнительных элементов для корректной работы. Необходимо подключить все компоненты, указанные в нижней части окна. В итоге в столбце Sel не должно остаться жёлтых секций.

После выбора всех необходимых компонентов и нажатия кнопки Ок откроется рабочая среда проекта (рис.4.4). В её левой части находится окно менеджера проекта Project. В нижней части расположено окно консоли сообщений компилятора Build Output. В центральной части расположено пространство текстового редактора, где можно открывать и редактировать текстовые файлы проекта.

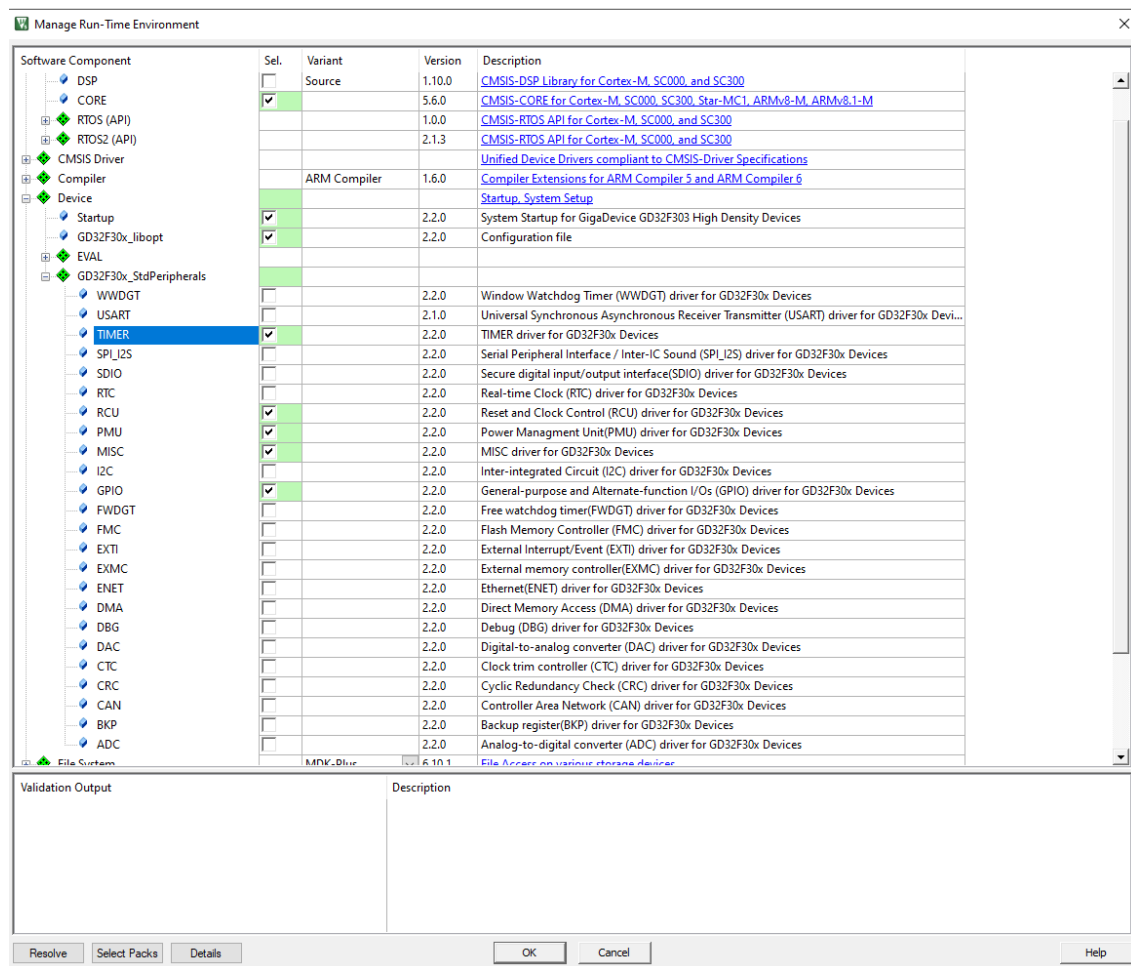


Рис. 4.3 Окно программного окружения

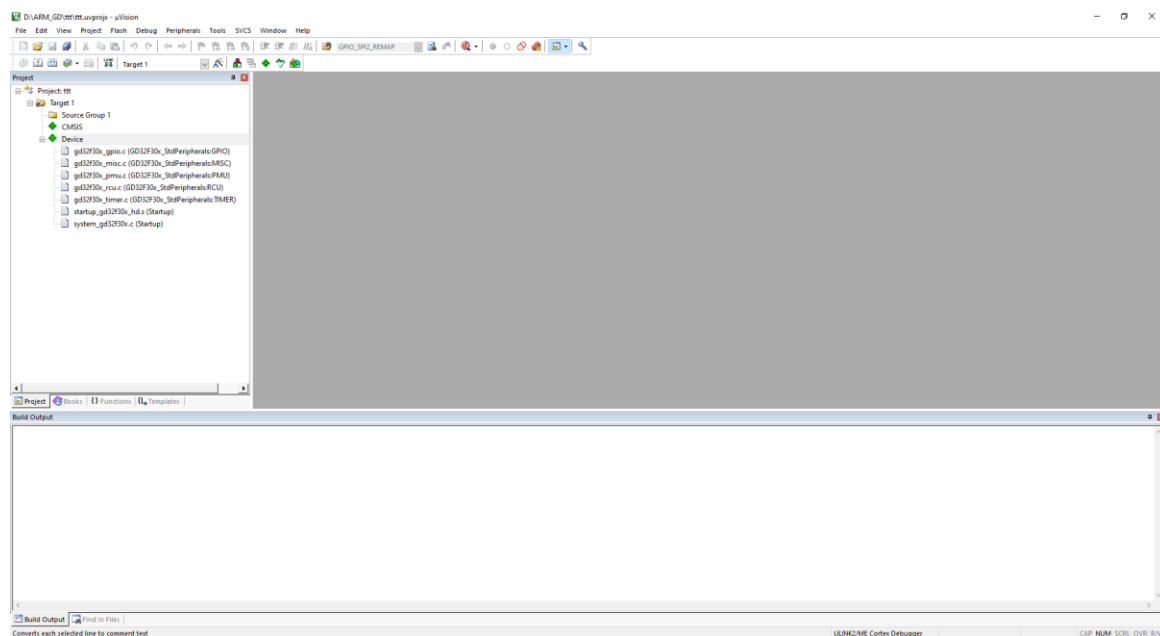


Рис. 4.4 Рабочая среда проекта

В менеджере проекта можно добавлять в него и удалять из него программные файлы (рис.4.5).

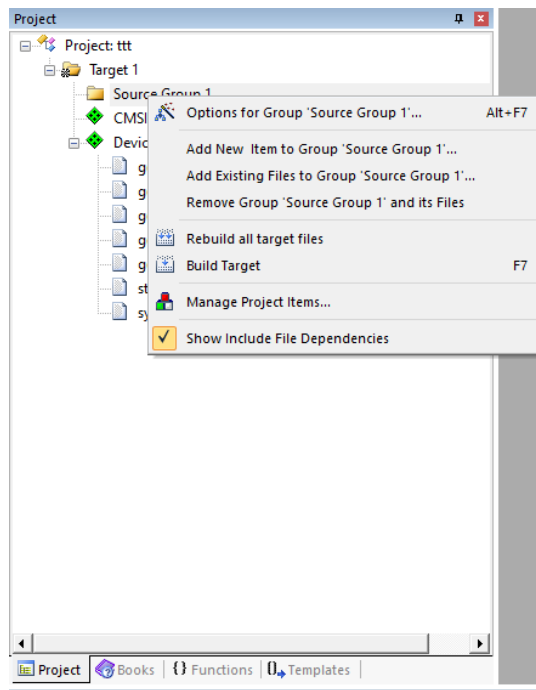


Рис. 4.5 Добавление и удаление файлов проекта

После создания проекта необходимо задать некоторые свойства целевой платы. Это делается выбором строки Target 1 и опции Options for Target 1 (рис.4.6).

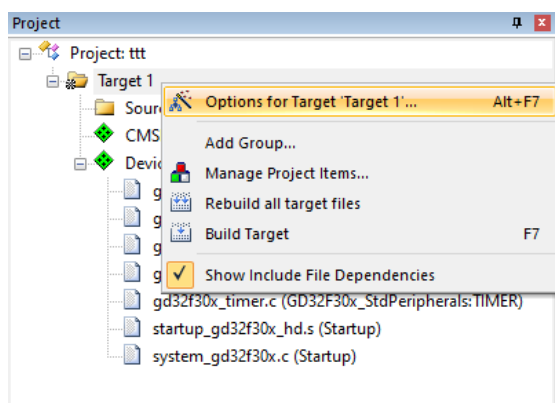


Рис. 4.6 Выбор опций целевой платы

После этого появляется окно опций целевой платы (рис.4.7). В графе Xtal необходимо задать частоту кварцевого резонатора синхрогенератора микроконтроллера, равную 25 МГц. Во вкладке Debug в графе Use необходимо выбрать тип отладочного адаптера CMSIS-DAP Adapter (рис.4.8).

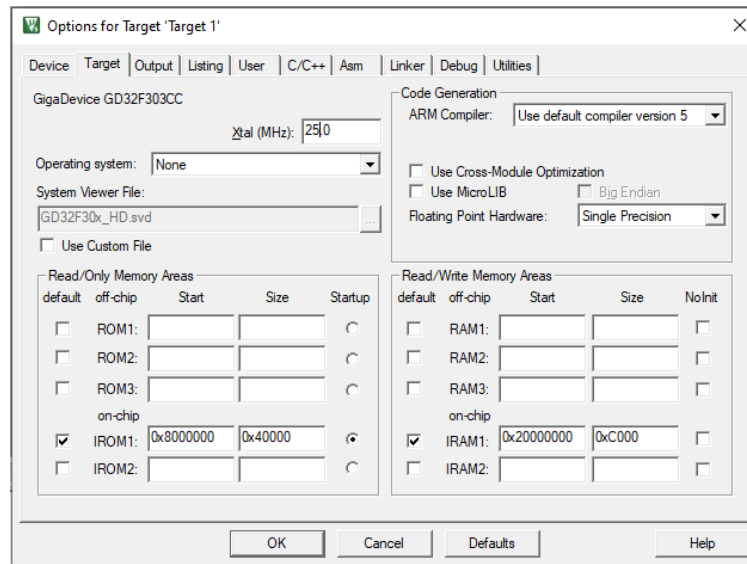


Рис. 4.7 Окно опций целевой платы

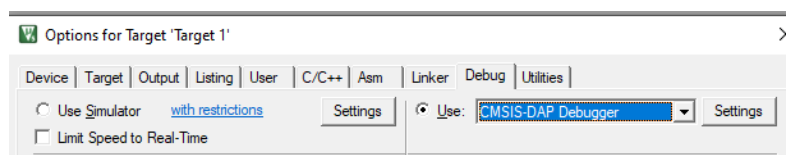


Рис. 4.8 Выбор отладочного адаптера

После создания всех файлов проекта их нужно откомпилировать и преобразовать в загрузочный модуль. Для этого нужно нажать на кнопку Rebuild (рис.4.9).

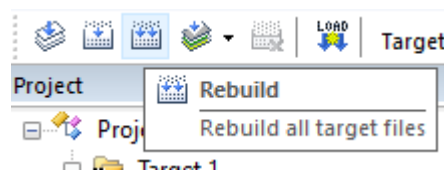


Рис. 4.9 Компиляция проекта

После успешной компиляции загрузочный модуль необходимо загрузить в целевую плату для отладки. Для этого нужно нажать кнопку Start/Stop Debug Session (рис.4.10). При этом происходит загрузка кода в память микроконтроллера.

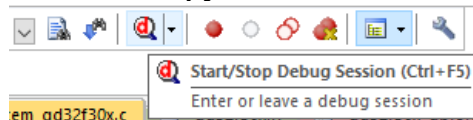


Рис. 4.10 Открытие отладочной сессии

При этом появляется окно отладочной сессии (рис.4.11).

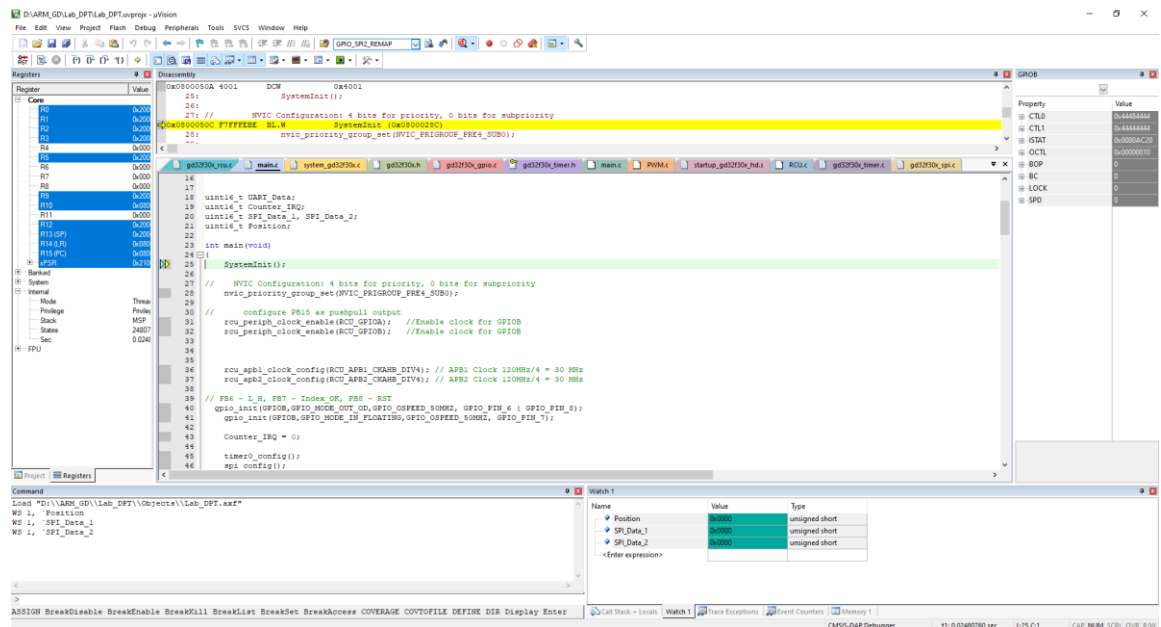


Рис. 4.11 Окно отладочной сессии

Для запуска программы на выполнение необходимо нажать кнопку Run (рис.4.12).

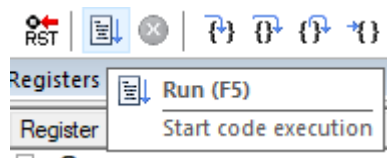


Рис. 4.12 Запуск программы на выполнение

Для пошаговой отладки программы предназначены кнопки Step, Step Over, Step Out, Run to Cursor Line (рис.4.12).

В левой части окна отладочной сессии выведено отображение регистров микроконтроллера (View- Registers Window). В правой части окна выведено отображение регистров периферийных устройств (Peripherals – System Viewer - ...). В правой нижней части окна выведено отображение программных переменных (View- Watch Window-...). Для установки точки останова программы нужно щёлкнуть левой кнопкой мыши справа в строке необходимой команды (рис.4.13).

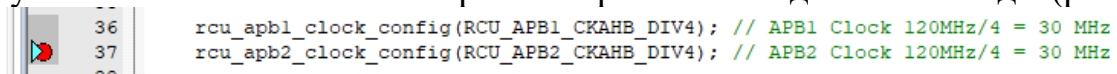


Рис. 4.13 Точка останова

Для завершения отладочной сессии нужно снова нажать кнопку Start/Stop Debug Session (рис.4.10).

5. Лабораторные работы

5.1. Работа №1 «Работа со светодиодным матричным дисплеем»

Для выполнения данной работы в материнскую плату вставляется дочерняя плата матричного дисплея, блок схема которой представлена на рисунке 5.1.

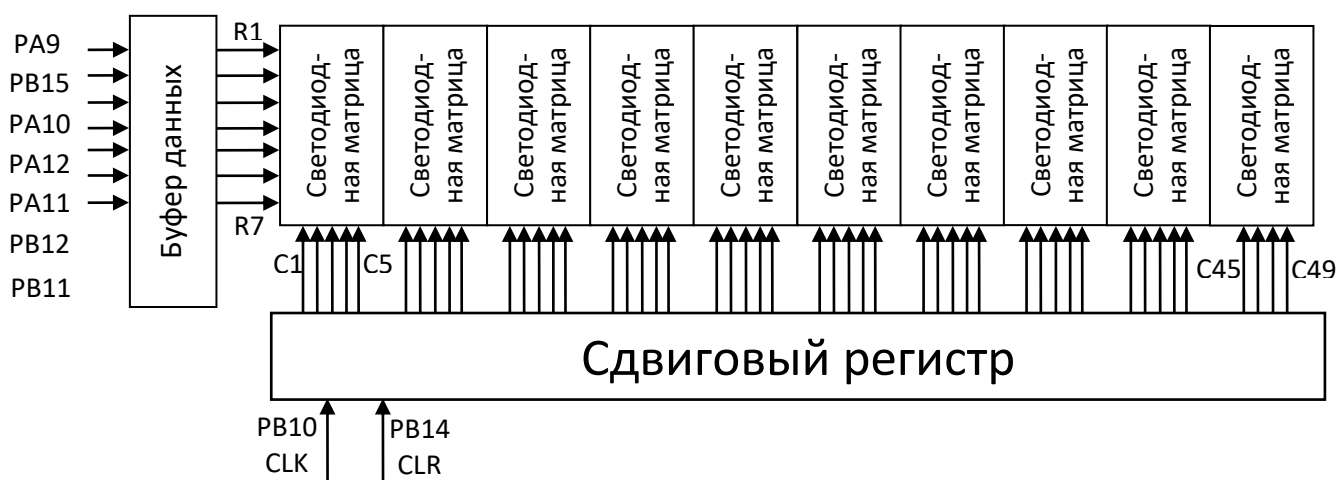


Рис. 5.1 Плата светодиодного экрана.

Матричный экран представляет собой светодиодную матрицу (на рис 5.2 представлен фрагмент из 5 столбцов) с 50 столбцами (используются только 49) и 7 строками. Линии 7 строк R1 – R7 подключены к портам PA9, PB15, PA10, PA12, PA11, PB12, PB11 микроконтроллера соответственно. Все эти порты необходимо инициализировать в конфигурации «открытый сток». Линии столбцов C1 – C49 подключены к выходам 49-разрядного сдвигового регистра. В выходном 49-разрядном коде сдвигового регистра в каждый момент времени единице равен только один разряд (активен только один столбец на светодиодной матрице).

По положительному перепаду входного сигнала CLK единица перемещается в следующий разряд выходного кода. После достижения единицей линии C49, следующим положительным перепадом сигнала CLK единица вновь перемещается на линию C1 и так далее. Сигнал CLK подключен к порту PB10 микроконтроллера. По положительному уровню входного сигнала CLR, подключенному к порту PB14 микроконтроллера, выходной код сдвигового регистра из любого предыдущего состояния переводится в начальное состояние, при котором единица установлена на

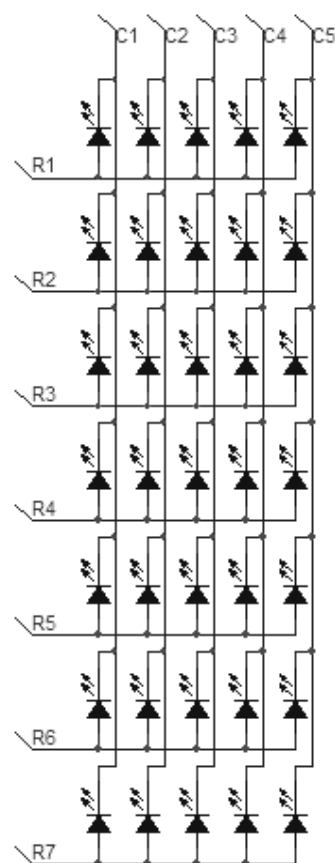


Рис. 5.2 Фрагмент светодиодной матрицы.

линии С1. Все эти порты необходимо инициализировать в конфигурации «открытый сток».

К каждой из линий столбцов подключены аноды семи светодиодов. Для того, чтобы светодиод светился необходимо, чтобы потенциал его анода был выше потенциала катода. Так как единица в каждый момент времени подается только на один из столбцов матрицы, светиться одновременно могут только светодиоды этого столбца. Причем светиться будут только те светодиоды активного столбца, катоды которых подключены к строкам, на которые в данный момент подан 0. Таким образом, активируя последовательно столбец за столбцом и выставляя синхронно с этим на линии строк соответствующую комбинацию сигналов, можно добиться динамического вывода требуемой информации с разверткой по столбцам. Для обеспечения статической картины без мерцания частота развертки всего кадра должна быть не менее 25 Гц, из чего следует, что минимальная частота смены активных столбцов должна быть не менее $49 \cdot 25 = 1225$ Гц.

Пример вывода текстовой информации на матричном дисплее представлен на рис.5.3.

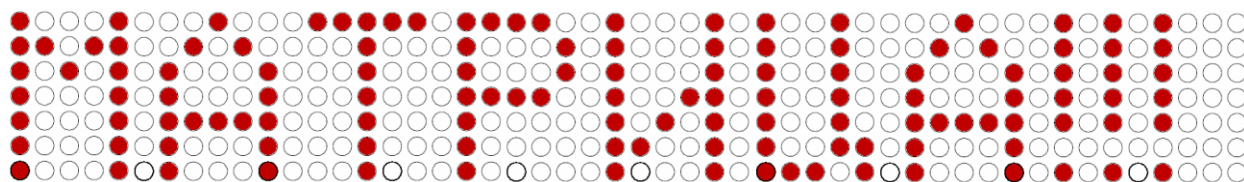


Рис. 5.3 Пример изображения на матричном дисплее.

5.2. Работа №2 «Контроль температуры»

Для выполнения данной работы в отладочную материнскую плату вставляется дочерняя плата термостата, блок схема которой представлена на рисунке 5.4.

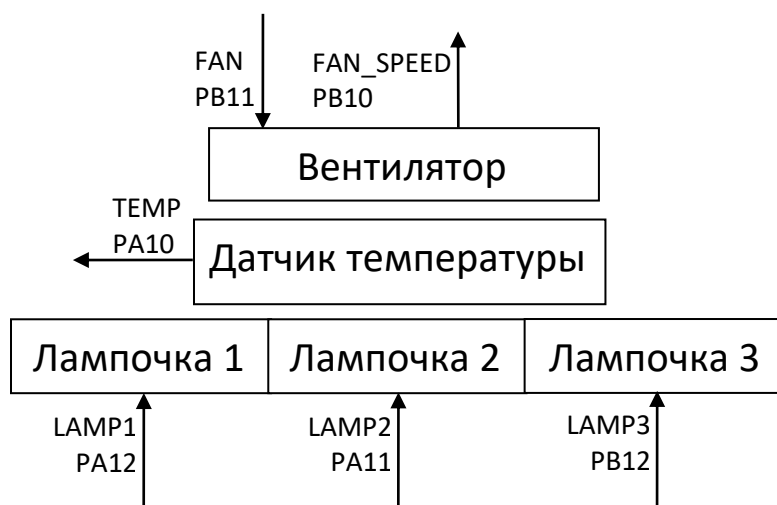


Рис. 5.4 Плата термостата.

На плате термостата размещены охладитель (вентилятор), нагреватели (3 лампочки накаливания) и датчик температуры. Датчик температуры вырабатывает выходной сигнал прямоугольной формы с изменяющимся периодом и длительностью импульса, форма которого представлена на рис.5.5. Там же представлена зависимость формы сигнала от измеряемой температуры. Для расчета температуры необходимо измерить длительность импульса и паузы в выходном сигнале датчика, который подключен к порту PA10 микроконтроллера.

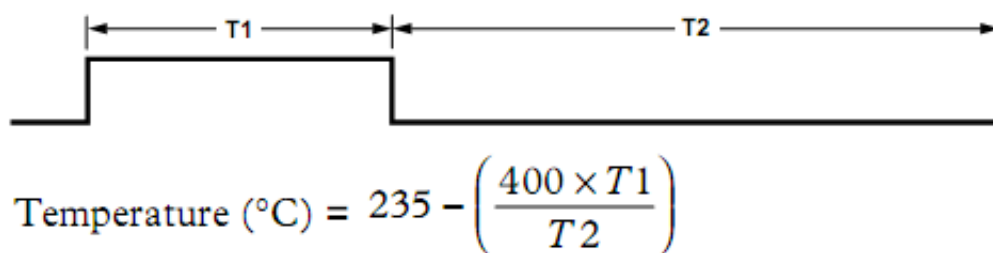


Рис. 5.5 Сигнал с датчика температуры.

Скорость вращения вентилятора определяется средним напряжением на его выводе питания. Для управления скоростью вращения вентилятора, питание на него подается через транзисторный ключ, управляющий вывод которого подключен к порту PB11 микроконтроллера (конфигурация – открытый сток). При нулевом уровне сигнала на выводе этого порта питание на вентилятор подается, а при единичном – нет. Таким образом, осуществляя широтно-импульсную модуляцию этого сигнала можно управлять средним уровнем напряжения питания вентилятора. Для полного выключения вентилятора необходимо постоянно поддерживать единичный уровень сигнала на выводе порта PB11 микроконтроллера. Текущая скорость вращения вентилятора может быть измерена посредством выхода FAN_SPEED, подключенного к порту PB10 микроконтроллера. На этом выводе при вращении вентилятора вырабатывается прямоугольный сигнал, частота которого пропорциональна частоте вращения. Для нагрева используются три лампочки накаливания, питание на которые подается через транзисторные ключи. Управление этими ключами осуществляется сигналами, подключенными к портам PA12, PA11 и PB12 микроконтроллера (конфигурация – открытый сток). Для включения любой из лампочек необходимо подать сигнал нулевого уровня на вывод соответствующего порта.

Комбинируя интервалы включенного и выключенного состояния нагревателей с включением и выключением вентилятора, можно обеспечивать требуемый температурный режим, который контролируется датчиком температуры.

5.3. Работы №3,4 «Управление электродвигателями»

Для выполнения данной работы в отладочную материнскую плату вставляется дочерняя плата управления электродвигателями, блок схема которой представлена на рисунке 5.6.

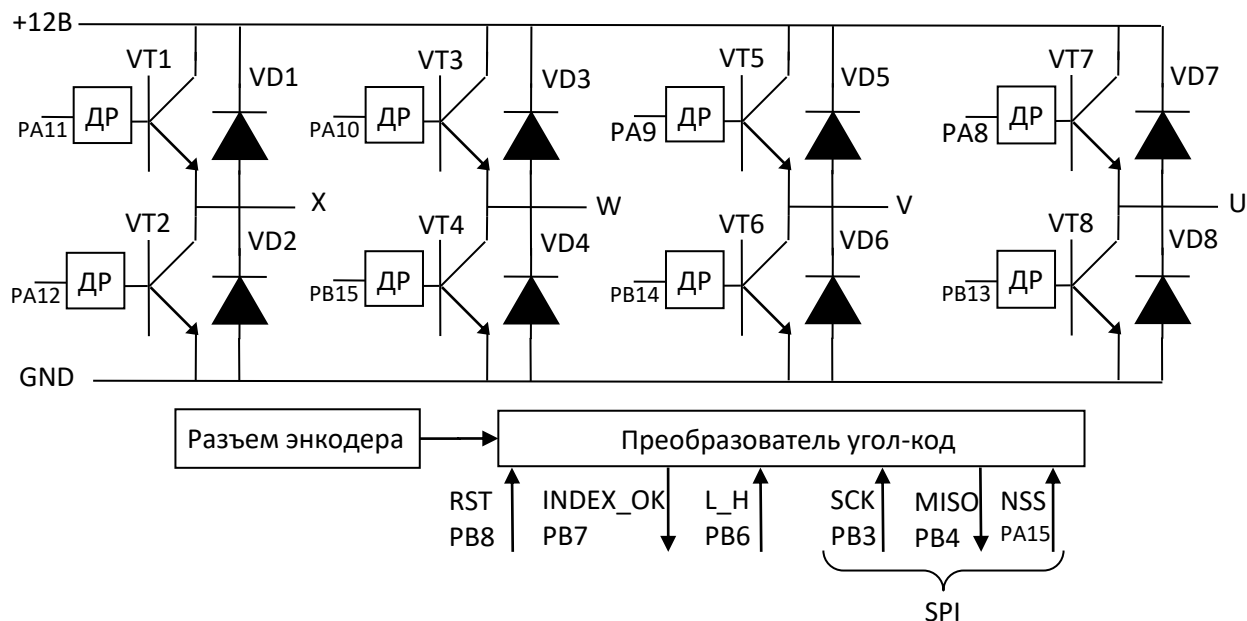


Рис. 5.6 Плата электропривода.

На этой плате реализован четырехфазный инвертор напряжения и преобразователь сигналов оптического энкодера, используемого в качестве датчика положения вала двигателя.

Сигналы управления ключами инвертора подключены к выводам портов PA11, PA12, PA10, PB15, PA9, PB14, PA8, PB13 микроконтроллера. Все эти порты необходимо инициализировать в конфигурации «открытый сток». Открытому состоянию транзисторного ключа соответствует логический «0» на входе соответствующего порта.

Преобразователь сигналов оптического энкодера связан с микроконтроллером посредством стандартного интерфейса SPI.

Интерфейс энкодера состоит из 6 линий:

- RST (подключен к выводу порта PB8, конфигурация – открытый сток) – сброс преобразователя энкодера в начальное состояние по единичному уровню сигнала;
- INDEX_OK (подключен к выводу порта PB7) – логическая единица на этом выводе появляется, когда преобразователь находит нулевую метку и начинает выдавать правильное абсолютное угловое положение;
- L_H (подключен к выводу порта PB6, конфигурация – открытый сток) – выбор передаваемого байта положения (высокий логический уровень – старший байт, низкий логический уровень – младший байт);
- SCK (подключен к выводу порта PB3, конфигурация – открытый сток) – линия тактовых импульсов синхронизации SPI;
- MISO (подключен к выводу порта PB4) – линия данных шины SPI;
- NSS (подключен к выводу порта PA15, конфигурация – открытый сток) – линия выбора устройства шины SPI.

Код угла имеет разрядность 10 бит, т.е. 1024 значения на один полный оборот вала двигателя. Так как SPI производит обмен только по 8 бит за одну посылку

ку, то код положения передается за два прохода. В зависимости от уровня на L_H , микросхема преобразователя передает либо младшие, либо старшие 8 бит кода, но из посылки старшего байта информацию несут только младшие 2 бита (см. рис 5.7). Начинать чтение данных необходимо всегда с младшего байта.

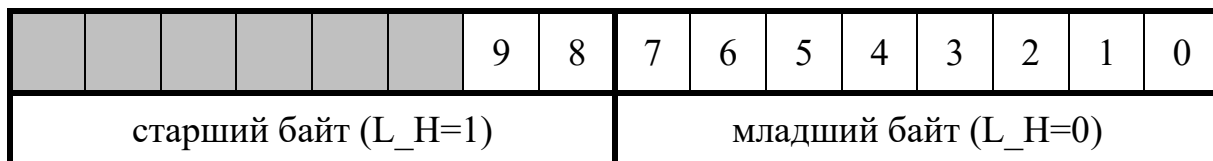


Рис. 5.7 Формат данных с преобразователя угол-код.

Для управления двигателем постоянного тока используются два плеча инвертора (в примере на рис.5.8 это плечи X и W), в диагональ которых включен якорь двигателя. Для того, чтобы двигатель вращался в одну сторону, к его якору должно быть приложено напряжение через открытые транзисторы $VT1$ и $VT4$. Транзисторы $VT2$ и $VT3$ в этом случае должны быть закрыты для исключения сквозных токов. Для вращения двигателя в противоположном направлении, должны быть открыты транзисторы $VT2$ и $VT3$, а $VT1$ и $VT4$ закрыты. При этом к якору двигателя прикладывается отрицательное напряжение. Регулировать скорость вращения вала двигателя можно, изменяя среднее значение напряжения на якоре двигателя, что осуществляется с помощью широтно-импульсной модуляции сигнала управления одного или обоих ключей открытой диагонали.

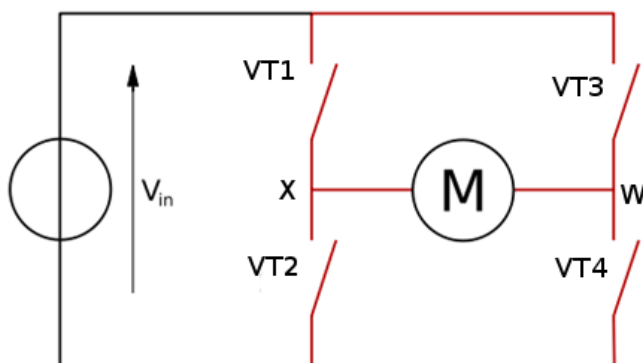


Рис. 5.8 Схема подключения ДПТ.

При управлении двухфазным шаговым двигателем используются все четыре плеча инвертора. При этом положение ротора определяется направлением протекания тока по каждой из обмоток. Схема подключения обмоток шагового двигателя к плате представлена на рис. 5.9.

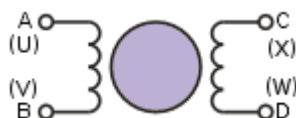


Рис. 5.9 Схема подключения шагового двигателя.

Существует несколько способов управления шаговым двигателем. Основные из них представлены на рис. 5.10.

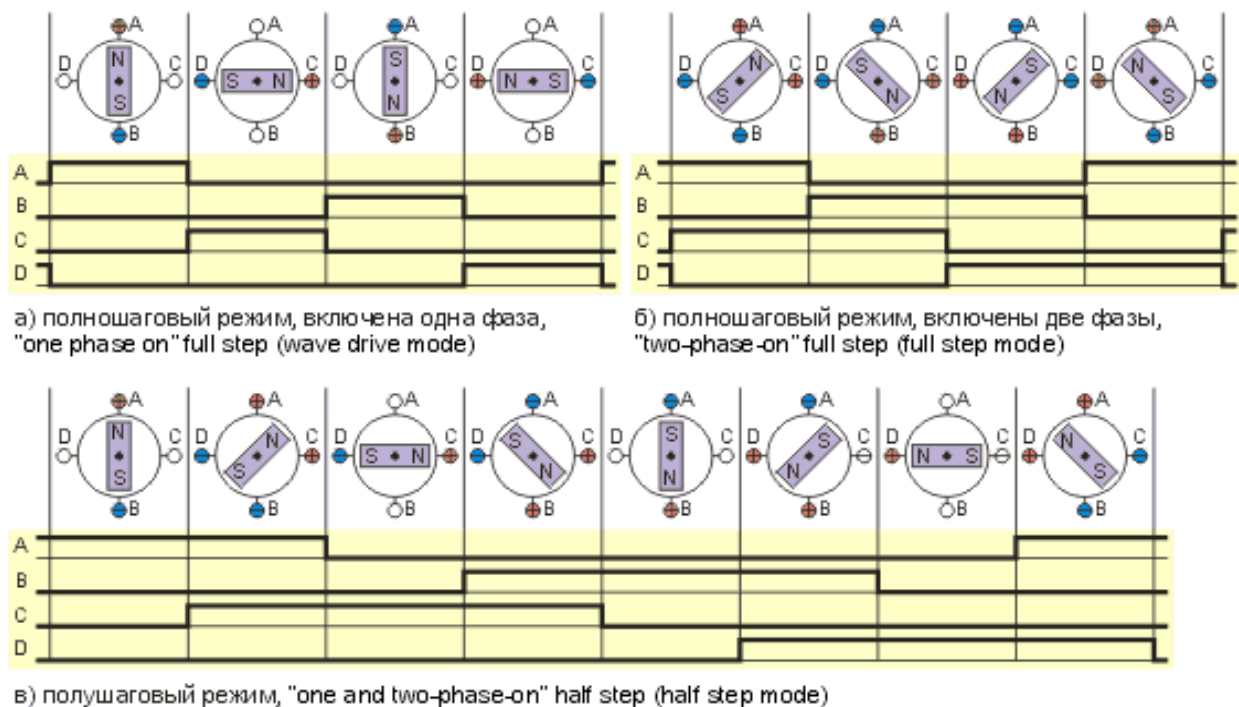


Рис. 5.10 Способы управления шаговым двигателем.

1) Попеременная коммутация фаз (одновременно включена только одна фаза).

Точки равновесия ротора для каждого шага совпадают с «естественными» точками равновесия ротора у двигателя с незапитанными фазами.

2) Управление фазами с перекрытием (одновременно включены две фазы).

При этом способе управления ротор фиксируется в промежуточных позициях между полюсами статора и обеспечивается примерно на 40% больший момент, чем в случае одной включенной фазы. Этот способ управления обеспечивает такой же угол шага, как и первый способ, но положение точек равновесия ротора смещено на полшага.

3) Полушаговый режим (каждый второй шаг запитана лишь одна фаза, а в остальных случаях запитаны две). В результате угловое перемещение ротора составляет половину угла шага для первых двух способов управления.

5.4. Работа №5 «Символьный экран и матричная клавиатура»

Для выполнения данной работы в отладочную материнскую плату вставляется дочерняя плата символьного экрана и матричной клавиатуры, структура которой представлена на рис.5.11.

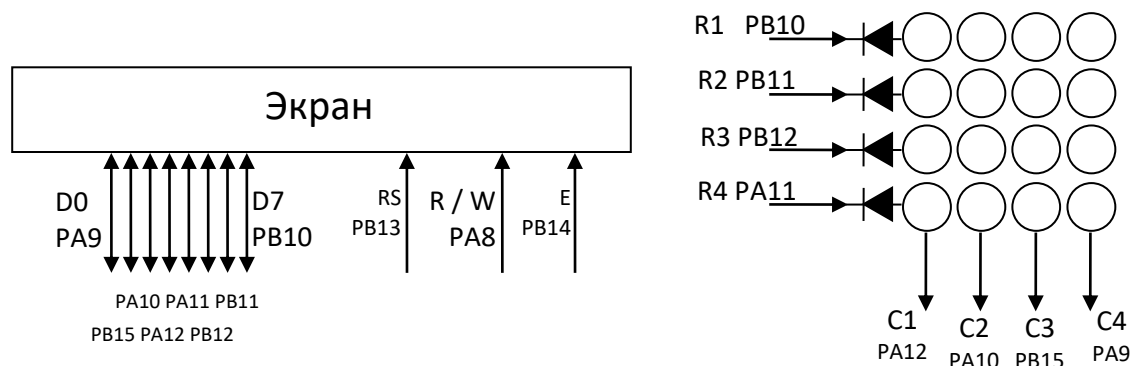


Рис. 5.11 Плата символьного экрана и матричной клавиатуры.

Клавиатура представляет собой матрицу из 16 клавиш, расположенных на пересечениях 4 строк и 4 столбцов, подключенных к портам микроконтроллера. Ее схема представлена на рис.5.12. В исходном состоянии на всех линиях столбцов поддерживается высокий логический уровень сигнала за счет подключенных к ним резисторов. При нажатии клавиши соответствующие строка и столбец замыкаются. Опрос клавиатуры осуществляется последовательным сканированием строк. Для этого на все линии строк кроме одной подаются сигналы высокого логического уровня, и только на одну, опрашиваемую в данный момент строку, подается сигнал нулевого уровня. Если ни одна из клавиш, подключенных к этой строке, не нажата, то на всех линиях столбцов будет присутствовать высокий логический уровень сигналов. Если же какие-либо клавиши, подключенные к этой строке, нажаты, то на соответствующих линиях столбцов будут присутствовать сигналы низкого логического уровня. Зная номер текущей опрашиваемой строки и номер столбца с нулевым уровнем сигнала, можно однозначно идентифицировать нажатую клавишу. В следующем цикле опроса сигнал нулевого уровня подается на следующую линию строки, и таким образом опрашиваются клавиши, подключенные к этой строке. Для опроса всей клавиатуры в данном случае необходимо четыре цикла опроса строк.

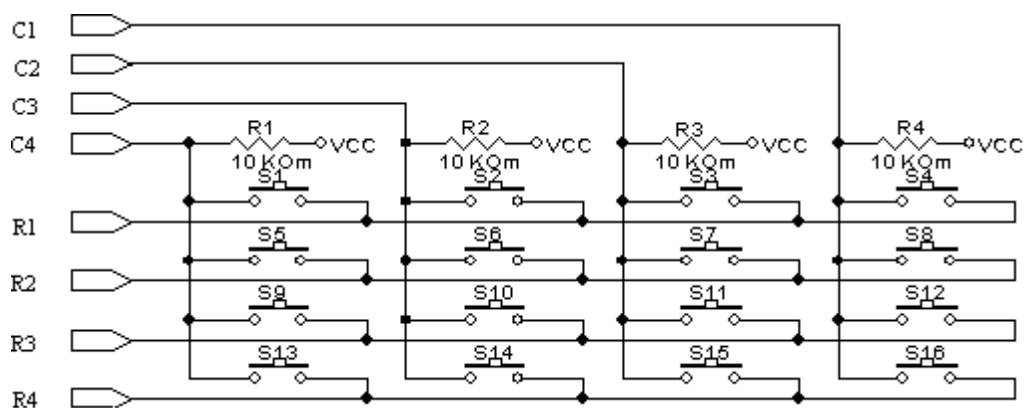


Рис. 5.12 Схема матричной клавиатуры.

Кроме того, на плате данной лабораторной работы размещён знакосинтезирующий текстовый буквенно-цифровой ЖК-дисплей. Экран имеет 2 строки по 16 символов в каждой. Размер шрифта 5*8 пикселей. Экран имеет встроенный контроллер управления и встроенное ОЗУ, в каждой ячейке которой хранится 8-битный код символа, выводимого на экран. Каждому знакоместу однозначно сопоставлена ячейка памяти. Все действия по выводу и регенерации изображения на экран выполняются внутренним микроконтроллером дисплея. Центральному управляющему микроконтроллеру для вывода информации на дисплей необходимо лишь записать коды выводимых символов во внутреннее ОЗУ дисплея и задать режим его работы с помощью соответствующих команд(см. Табл. 5.3).

Встроенное ОЗУ экрана делится на две независимые области, с разным назначением. Первая область – DDRAM (Display Data RAM) – та часть памяти, в которой хранятся ASCII-коды выводимых на экран символов. В таблице символов, изображения которых записаны в ПЗУ встроенного в дисплей контроллера есть 8 кодов символов предназначенных для вывода пользовательских знаков, графические изображения которых в специальном виде (см рисунок 5.13) хранятся во второй области ОЗУ – CGRAM (Character Generator RAM). Размер CGRAM составляет 64 байта (коды от 0x00 до 0x3F)

Character Codes (DDRAM data)								CGRAM Address								Character Patterns (CGRAM data)															
7	6	5	4	3	2	1	0	5				4				3				2				1				0			
High				Low				High				Low				High				Low				High				Low			
0 0 0 0 * 0 0 0 																															

Рис. 5.13 Формат хранения пользовательских знаков в CGRAM

Размер DDRAM составляет 127 байт (коды от 0x00 до 0x7F). Это сделано в целях унификации, т.к. существуют экраны с различным количеством строк и символов в строке, и во все типы экранов устанавливаются одинаковые типы кон-

троллеров. Для данного экрана соответствие знакомест на дисплее кодам в DDRAM представлено ниже:

№ символа в строке	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
адрес в DDRAM	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F
	40	41	42	43	44	45	46	47	48	49	4A	4B	4C	4D	4E	4F

Обмен данными с дисплеем осуществляется по параллельной магистрали, в которую входят:

- шина данных D0–D7 (сигналы подключены к выводам портов PA9, PB15, PA10, PA12, PA11, PB12, PB11, PB10 микроконтроллера) - двунаправленная шина, по которой передаются данные в циклах обмена информацией с дисплеем. Все эти порты необходимо инициализировать в конфигурации «открытый сток»;
- сигнал RS (подключен к выводу порта PB13 микроконтроллера, конфигурация – открытый сток) – единичный уровень его означает запись(чтение) кода символа, который будет выведен(или который уже выведен) на текущее знакоместо, а нулевой - запись команды управления в дисплей (включение отключения экрана, отображение курсора, установка адреса текущего знакоместа) или чтение текущего состояния экрана (бит готовности);
- сигнал R/W (подключен к выводу порта PA8 микроконтроллера, конфигурация – открытый сток) – его единичный уровень означает чтение из дисплея, а нулевой - запись в дисплей;
- сигнал E(подключен к выводу порта PB14 микроконтроллера, конфигурация – открытый сток) – строб чтения/записи – в этом сигнале формируется нулевой импульс при реализации чтения либо записи информации в или из дисплея.

Диаграмма цикла записи информации в дисплей представлена на Рис. 5.14, а её временные параметры в Таблица 5.1.

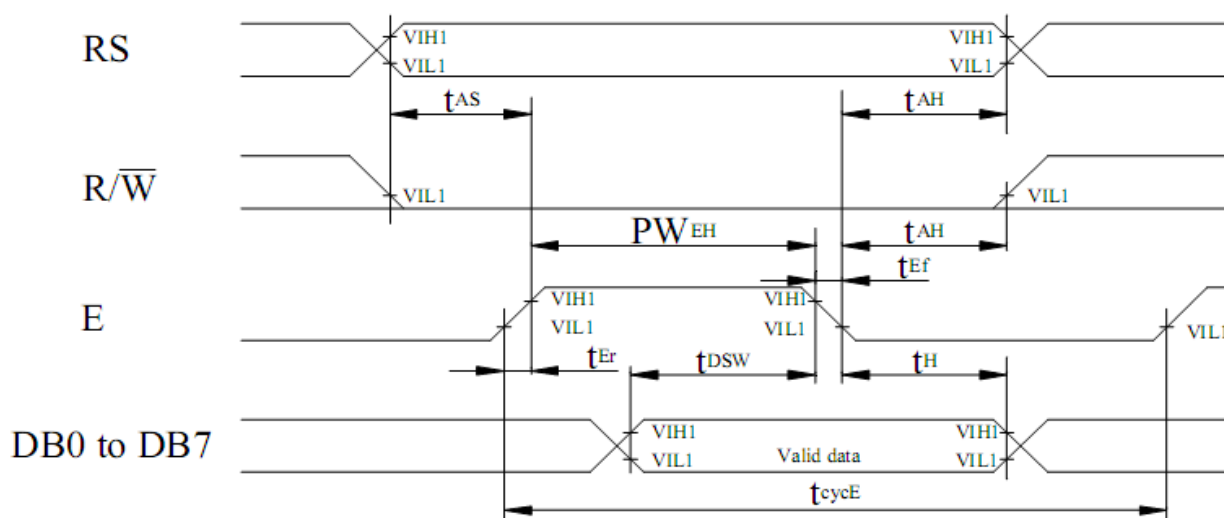


Рис. 5.14 Цикл записи информации в дисплей.

Таблица 5.1 Временные параметры диаграммы цикла записи.

Величина	Обозначение	Значение		Ед. изм.
		Мин.	Макс.	
Строб E:				
время цикла	t_{cycE}	400	-	нс
ширина импульса высокого уровня	PW_{EH}	150	-	нс
время нарастания/спада	$t_{\text{Er}}, t_{\text{Ef}}$	-	25	нс
Время установления стробов адреса (от RS, R/W к E)	t_{AS}	30	-	нс
Время удержания стробов адреса	t_{AH}	10	-	нс
Время установления сигналов на линиях данных	t_{DSW}	40	-	нс
Время удержания сигналов на линиях данных	t_{H}	10	-	нс

Диаграмма цикла чтения информации из дисплея представлена на Рис. 5.15, а её временные параметры в

Табл. 5.2Таблица 5.1.

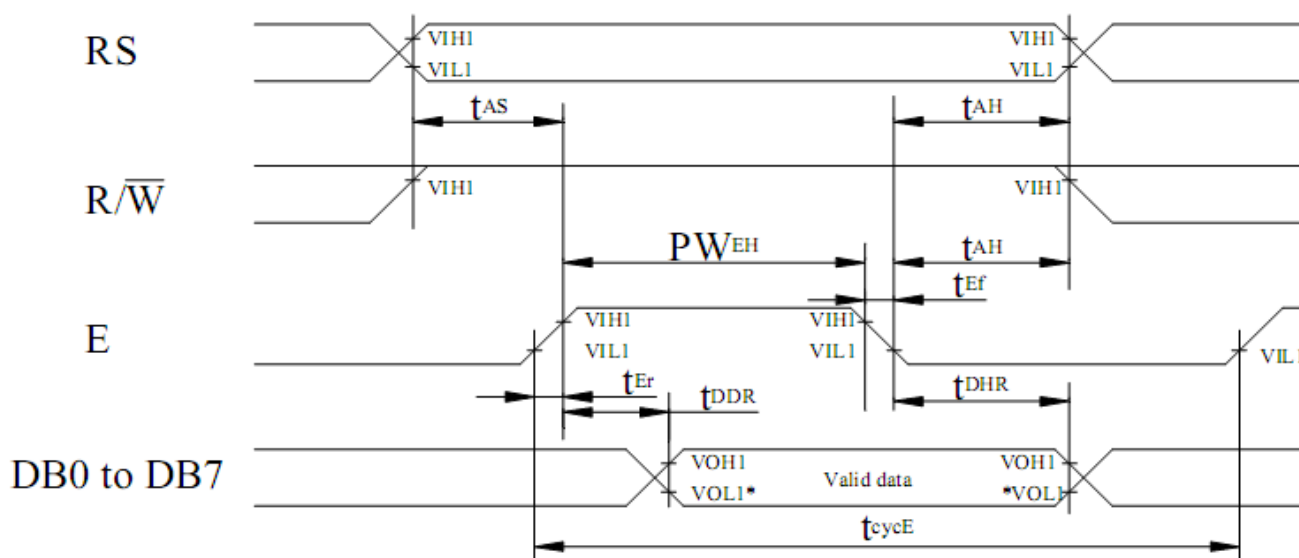


Рис. 5.15 Цикл чтения информации из дисплея.

Табл. 5.2 Временные параметры диаграммы цикла чтения.

Величина	Обозначение	Значение		Ед. изм.
		Мин.	Макс.	
Строб E:				
время цикла	t_{cycE}	400	-	нс
ширина импульса высокого уровня	PW_{EH}	150	-	нс
время нарастания/спада	$t_{\text{Er}}, t_{\text{Ef}}$	-	25	нс
Время установления стробов адреса (от RS, R/W к E)	t_{AS}	30	-	нс
Время удержания стробов адреса	t_{AH}	10	-	нс
Задержка перед установлением сигналов на линиях данных	t_{DDR}	-	100	нс

Время удержания сигналов на линиях данных	t_{DHR}	20	-	нс
---	-----------	----	---	----

Табл. 5.3 Управляющие команды дисплея.

Команда	Код команды										Описание	Время выполнения
	RS	R/W	DB 7	DB 6	DB 5	DB 4	DB 3	DB 2	DB 1	DB 0		
Очистка экрана	0	0	0	0	0	0	0	0	0	1	Записывает во все ячейки зна-комест значение 00h и уста-навливает указатель адреса текущего знакоместа в 00h, те первый символ первой строки.	1.53 мс
Возврат курсора в начало	0	0	0	0	0	0	0	0	1	-	Устанавливает указатель адре-са текущего знакоместа в 00h, те первый символ первой строки. Содержимое памяти не меняется.	1.53 мс
Установка режима записи	0	0	0	0	0	0	0	1	I/D	SH	<u>I/D</u> : режим смещения указате-ля адреса текущего знакоместа после записи очередного сим-вола. «1» - счетчик увеличивается (курсор сдвигается впра-во), «0» - счетчик уменьшается (курсор сдвигается влево); <u>S</u> : режим сдвига всего содер-жимого экрана после записи очередного символа вместо сдвига курсора. «0» - нет сдвига изображе-ния, «1» - изображение сдвига-ется в зависимости от I/D.	39 мкс
Выбор режима отображения	0	0	0	0	0	0	1	D	C	B	<u>D</u> : изображения на экране «0» - выключено, «1» - включено; <u>C</u> : курсор в виде подчеркива-ния «0» - выключен, «1» - включен; <u>B</u> : курсор в виде мерцающего знакоместа «0» - выключен, «1» - включен.	39 мкс
Сдвиг курсора или экрана	0	0	0	0	0	1	S/C	R/L	-	-	Команда, производящая опе-рацию сдвига содержимого экрана (без изменений в DDRAM) или курсора в зави-симости от значения флагов <u>S/C</u> : «0» - сдвиг курсора, «1» - сдвиг экрана; <u>R/L</u> : «0» - влево, «1» - вправо.	39 мкс

Установка параметров развертки и шины данных	0	0	0	0	1	DL	N	F	-	-	DL: ширина шины данных «0» - 4 разряда, «1» - 8 разрядов; N: режим развертки изображения «0» - одна строка, «1» - две строки; F: размер матрицы символов «0» - 5x8 точек, «1» - 5x10 точек.	39 мкс
Установка адреса CGRAM	0	0	0	1	AC5	AC4	AC3	AC2	AC1	AC0	Присвоение счетчику AC адреса в области CGRAM.	39 мкс
Установка адреса DDRAM	0	0	1	AC6	AC5	AC4	AC3	AC2	AC1	AC0	Присвоение счетчику AC адреса в области DDRAM.	39 мкс
Чтение флага занятости и указателя адреса	0	1	BF	AC6	AC5	AC4	AC3	AC2	AC1	AC0	Флаг занятости может быть опрошен в любой момент, даже во время выполнения команды. Так же эта команда возвращает текущее значение счетчика адреса AC.	0 мкс
Запись данных в ОЗУ	1	0	D7	D6	D5	D4	D3	D2	D1	D0	Запись данных в текущую ячейку ОЗУ (DDRAM/CGRAM)	43 мкс
Чтение данных из ОЗУ	1	1	D7	D6	D5	D4	D3	D2	D1	D0	Чтение данных из текущей ячейки ОЗУ (DDRAM/CGRAM)	43 мкс

После подачи питания дисплей необходимо инициализировать. Блок схема алгоритма инициализации представлена на Рис. 5.16.

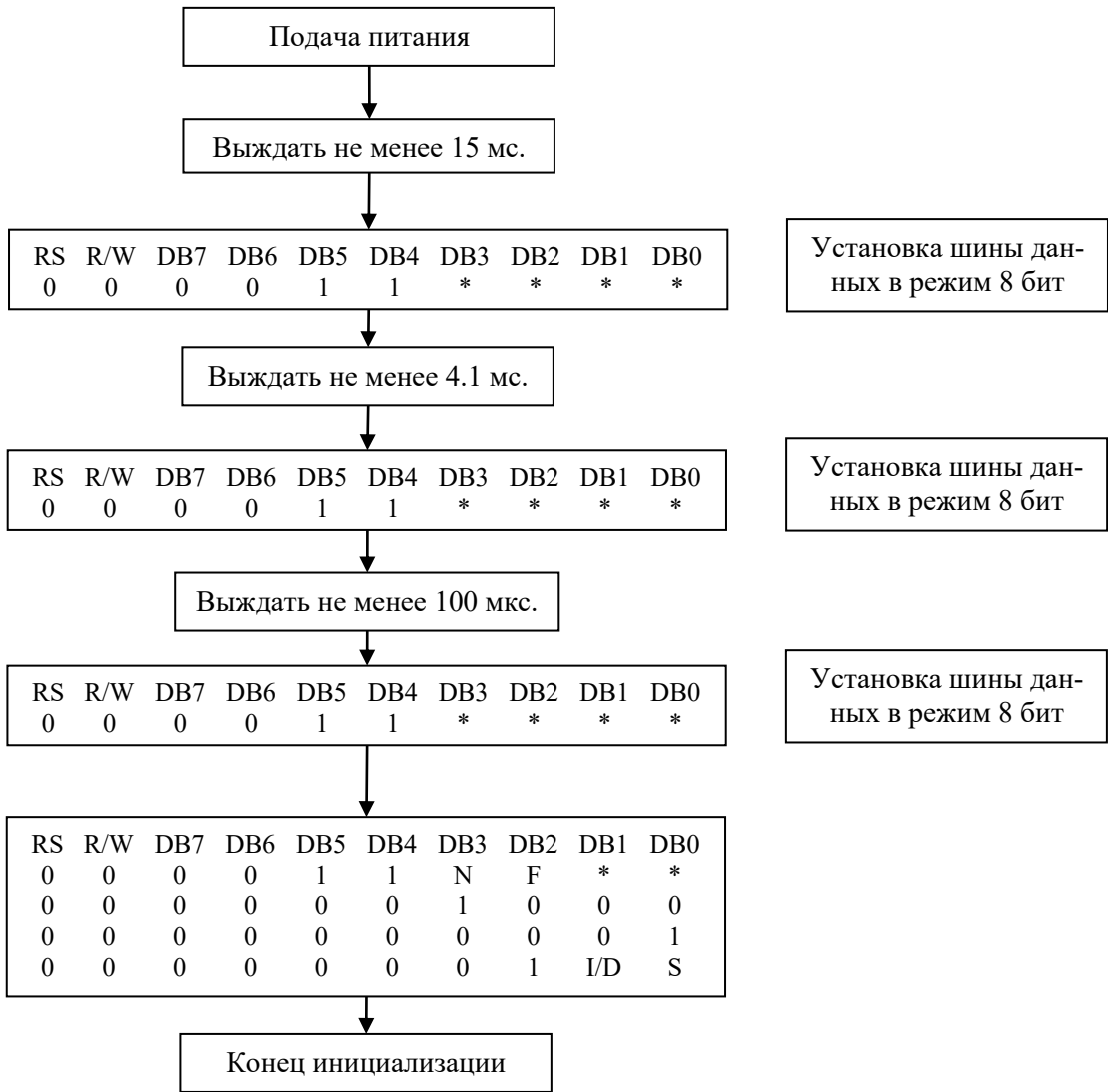


Рис. 5.16 Блок-схема алгоритма инициализации

Символы, отображаемые дисплеем и соответствующие им кодов, записываемые в память дисплея, представлены в Табл. 5.4 Таблица.

Табл. 5.4 Таблица символов, отображаемых дисплеем.

Upper 4 bit Lower 4 bit		LLLL	LLH	LLHL	LLHH	LHLL	LHLH	LHHL	LHHH	HLLL	HLLH	HLHL	HLHH	HHLL	HHLH	HHHL	HHHH
LLLL	CG RAM (1)				0	1	2	3	4			5	6	7	8	9	A
LLH	CG RAM (2)			!	1	A	D	a	q			Г	Я	u	,	U	М
LLHL	CG RAM (3)			"	2	B	R	b	r			Е	Б	ь	u	Ш	К
LLHH	CG RAM (4)			#	3	C	S	c	s			К	В	ы	u	а	А
LHLL	CG RAM (5)			\$	4	O	T	t				Э	Г	ь	у	Ф	В
LHLH	CG RAM (6)			%	5	E	U	e	u			Н	Б	а	х	U	·
LHHL	CG RAM (7)			&	6	F	V	v				Х	W	W	у	W	9
LHHH	CG RAM (8)			'	7	G	W	w				Г	Б	а	I	·	Б
HLLL	CG RAM (1)			(	8	H	X	x				П	М	а	U	·	*
HLLH	CG RAM (2)			)	9	I	V	i	v			У	В	а	↑	·	·
HLHL	CG RAM (3)			*	0	J	Z	j	z			Ф	К	а	↓	а	·
HLHH	CG RAM (4)			+	1	K	L	k	l			Ч	Г	а	W	а	*
HHLL	CG RAM (5)			,	<	L	*	l	1			U	W	W	W	U	U
HHLH	CG RAM (6)			-	=	M	J	m	1			Б	W	а	W	W	а
HHHL	CG RAM (7)			.	>	N	^	n	e			В	W	а	у	а	W
HHHH	CG RAM (8)			/	?	O	_	o	e			Э	Т	а	·	а	W

6. Описание встроенных периферийных устройств МК

6.1. Порты ввода/вывода.

Для доступа к аналоговым и цифровым ресурсам МК используются 35 внешних линий ввода/вывода. В МК GD32F303CC внешние линии ввода/вывода организованы как два 16-разрядных порта PA и PB и еще один неполный порт PC. Все порты ввода - вывода тактируются сигналом синхронизации периферийной шины APB2. Базовая структура порта представлена на рис.6.1.1.

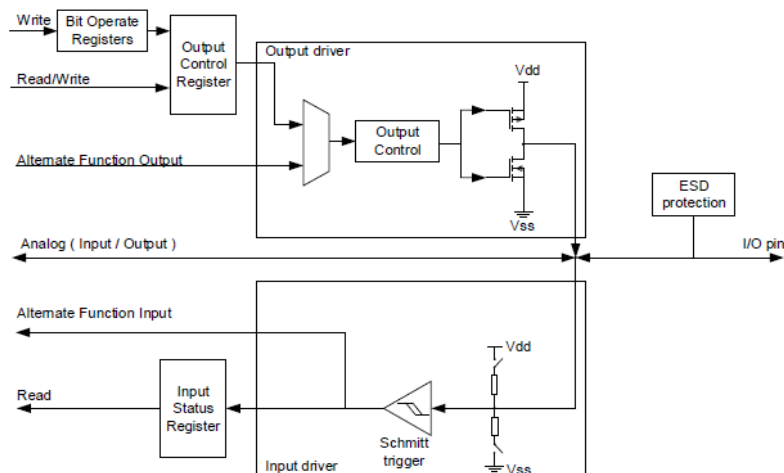


Рис.6.1.1 Структура порта ввода/вывода.

Каждый из выводов портов можно определить как порт ввода/вывода общего назначения (GPIO) или вход/выход определённого периферийного устройства. Следует иметь в виду, что состояние на внешнем выводе порта всегда можно прочитать из соответствующего регистра GPIOx_ISTAT порта независимо от его настройки. Каждый порт, используемый как GPIO, настраивается на выполнение функции входа или выхода. Требуемое значение выходного порта записывается в регистр GPIOx_OTL. Выходной каскад такого порта может иметь структуру «Push-Pull» (ключ с двумя транзисторами) или «Open Drain» (ключ только с нижним транзистором). Для входного порта в случае необходимости могут быть включены резисторы «подпорки» на ноль (Pull Down) или напряжение питания (Pull Up).

Конфигурация порта осуществляется с помощью функции
`void gpio_init(uint32_t gpio_periph, uint32_t mode, uint32_t speed, uint32_t pin),`
где

gpio_periph - имя порта GPIOx (x = A,B,C);

mode – режим работы:

GPIO_MODE_AIN: analog input mode

GPIO_MODE_IN_FLOATING: floating input mode

GPIO_MODE_IPD: pull-down input mode

GPIO_MODE_IPU: pull-up input mode

GPIO_MODE_OUT_OD: GPIO output with open-drain

GPIO_MODE_OUT_PP: GPIO output with push-pull

GPIO_MODE_AF_OD: AFIO output with open-drain

GPIO_MODE_AF_PP: AFIO output with push-pull;

Speed – максимальная частота переключения порта, определяющая время фронта:

GPIO_OSPEED_10MHZ: output max speed 10MHz
GPIO_OSPEED_2MHZ: output max speed 2MHz
GPIO_OSPEED_50MHZ: output max speed 50MHz
GPIO_OSPEED_MAX: output max speed more than 50MHz;

Pin – номер порта GPIO_PIN_x(x=0..15).

Изменение альтернативной функции вывода осуществляется с помощью функции `gpio_pin_remap_config(uint32_t remap, ControlStatus newvalue)`, где

Remap – тип реконфигурируемого периферийного устройства;

Newvalue - ENABLE or DISABLE.

Установка выходного порта в заданное состояние осуществляется с помощью функций

`void gpio_bit_set(uint32_t gpio_periph, uint32_t pin);`

`void gpio_bit_reset(uint32_t gpio_periph, uint32_t pin)`, где

gpio_periph - имя порта GPIOx(x = A,B,C);

Pin – номер порта GPIO_PIN_x(x=0..15).

Чтение состояния входного порта осуществляется с помощью функции

`FlagStatus gpio_input_bit_get(uint32_t gpio_periph, uint32_t pin)`, где

gpio_periph - имя порта GPIOx(x = A,B,C);

Pin – номер порта GPIO_PIN_x(x=0..15).

FlagStatus - SET или RESET.

Для того, чтобы порты ввода/вывода функционировали необходимо предварительно включить их синхронизацию. Для включения синхронизации портов ввода/вывода используется функция

`void rcu_periph_clock_enable(rcu_periph_enum periph)`, где

periph - RCU_GPIOx (x=A,B,C).

6.2. Таймеры/счетчики

МК GD32F303CC имеет в своём составе 14 16-разрядных таймеров, каждый из которых имеет от 1 до 4 каналов ввода-вывода. Таймеры 1 – 6, 11 – 13 тактируются сигналом синхронизации периферийной шины APB1. Таймеры 1 – 6, 11 – 13 тактируются сигналом синхронизации периферийной шины APB1, а таймеры 0, 7 – 10 - сигналом синхронизации периферийной шины APB2.

Структура улучшенного таймера T0, T7 представлена на рис.6.2.1.

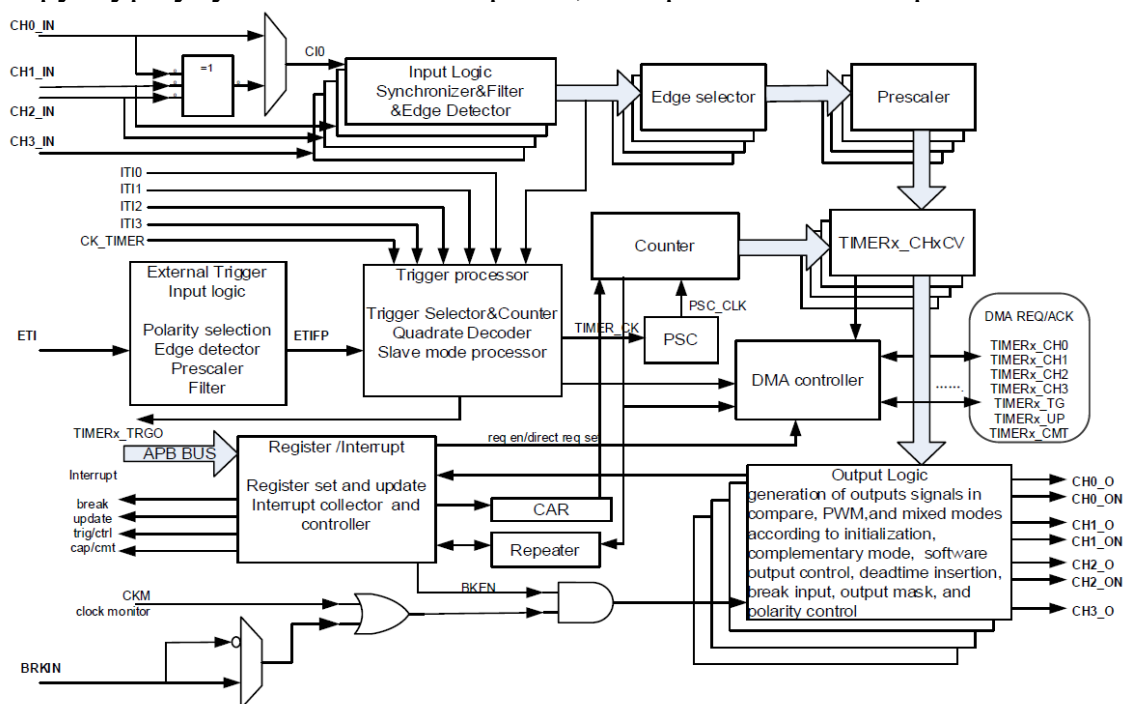


Рис.6.2.1

Таймеры могут быть использованы для отработки или измерения временных заданных интервалов. Каждый таймер имеет до 4 каналов захвата – сравнения для реализации функций быстрого ввода вывода.

Для инициализации таймера в программе необходимо объявить структуру типа `timer_parameter_struct`

```
typedef struct
{
    uint16_t prescaler;           /*!< prescaler value */
    uint16_t alignedmode;        /*!< aligned mode */
    uint16_t counterdirection;   /*!< counter direction */
    uint16_t clockdivision;      /*!< clock division value */
    uint32_t period;             /*!< period value */
    uint8_t repetitioncounter;   /*!< the counter repetition value */
} timer_parameter_struct;
```

В поле `prescaler` записывается уменьшенный на 1 коэффициент деления частоты синхронизации таймера.

В поле `alignedmode` записывается параметр, определяющий алгоритм работы таймера:

`TIMER_COUNTER_EDGE` – счёт в одну сторону;

`TIMER_COUNTER_CENTER_DOWN` – счёт в обе стороны, прерывание сравнения при счёте вниз;

`TIMER_COUNTER_CENTER_UP` – счёт в обе стороны, прерывание сравнения при счёте вверх;

`TIMER_COUNTER_CENTER_BOTH` – счёт в обе стороны, прерывание сравнения при счёте вверх и вниз;

В поле `counterdirection` записывается параметр, определяющий начальное направление счёта:

`TIMER_COUNTER_UP` – счёт вверх;

`TIMER_COUNTER_DOWN` – счёт вниз.

В поле `period` записывается уменьшенное на 1 значение перезагрузки таймера, определяющее диапазон его счёта. Когда таймер при счёте вверх досчитывает до этого значения по следующему перепаду сигнала синхронизации в него записывается ноль. Когда таймер при счёте вниз досчитывает до нуля по следующему перепаду сигнала синхронизации в него записывается это значение. В обоих случаях это вызывает запрос прерывания события обновления таймера `TIMER_INT_UP` (update event). Этот запрос прерывания должен быть сброшен программно с помощью функции

`timer_interrupt_flag_clear(TIMERX, TIMER_INT_UP)`.

В таймере может быть включён зеркальный регистр значения перезагрузки с помощью функции

`timer_auto_reload_shadow_enable(TIMERX)`.

При выключенном зеркальном регистре новое значение перезагрузки вступает в силу сразу после записи. При включенном зеркальном регистре новое значение перезагрузки вступает в силу после очередного события перезагрузки.

В поле `clockdivision` записывается параметр определяющий коэффициент деления частоты синхронизации для функций цифровой фильтрации и формирования «мёртвого времени» `fDTS`:

`TIMER_CKDIV_DIV1` - коэффициент деления частоты равен 1;

`TIMER_CKDIV_DIV2` - коэффициент деления частоты равен 1;

`TIMER_CKDIV_DIV4` - коэффициент деления частоты равен 4.

В поле `repetitioncounter` записывается уменьшенное на 1 значение коэффициента прореживания генерации события обновления таймера.

Инициализация таймера осуществляется с помощью функции

`void timer_init(uint32_t timer_periph, timer_parameter_struct* initpara),`

где

`timer_periph` – номер таймера `TIMER0` – `TIMER13`;

`initpara` – указатель на структуру инициализации таймера.

Пример инициализации таймера приведён ниже.

`timer_parameter_struct timer_initpara;`

```

timer_deinit(TIMERO);

// Timer reset

/* TIMERO configuration */
timer_initpara.prescaler      = 59;
// Timer Prescaler = 59 + 1 = 60 (60 MHz (APB2*2)/60
= 1 MHz)
timer_initpara.alignedmode    = TIMER_COUNTER_CENTER_BOTH;
// Count up and down
timer_initpara.counterdirection = TIMER_COUNTER_UP;
// Initial count up
timer_initpara.period         = 99;
// Reload value 100 (1MHz/100 = 10 KHz.
timer_initpara.clockdivision   = TIMER_CKDIV_DIV4;
// Deadtime and filter clock = Timer Clock/4 (120 MHz/4 = 30 MHz)
timer_initpara.repetitioncounter = 0;
// No repetition
timer_init(TIMERO,&timer_initpara);

```

Функция timer_deinit осуществляет начальную деинициализацию таймера.

Таким образом, при инициализации таймера задаются частота счётных импульсов, направление и предел счёта, частота синхронизации цифрового фильтра на входах каналов, коэффициент прореживания генерации события обновления таймера.

Диаграмма работы таймера при счёте вверх представлена на рис.6.2.2.

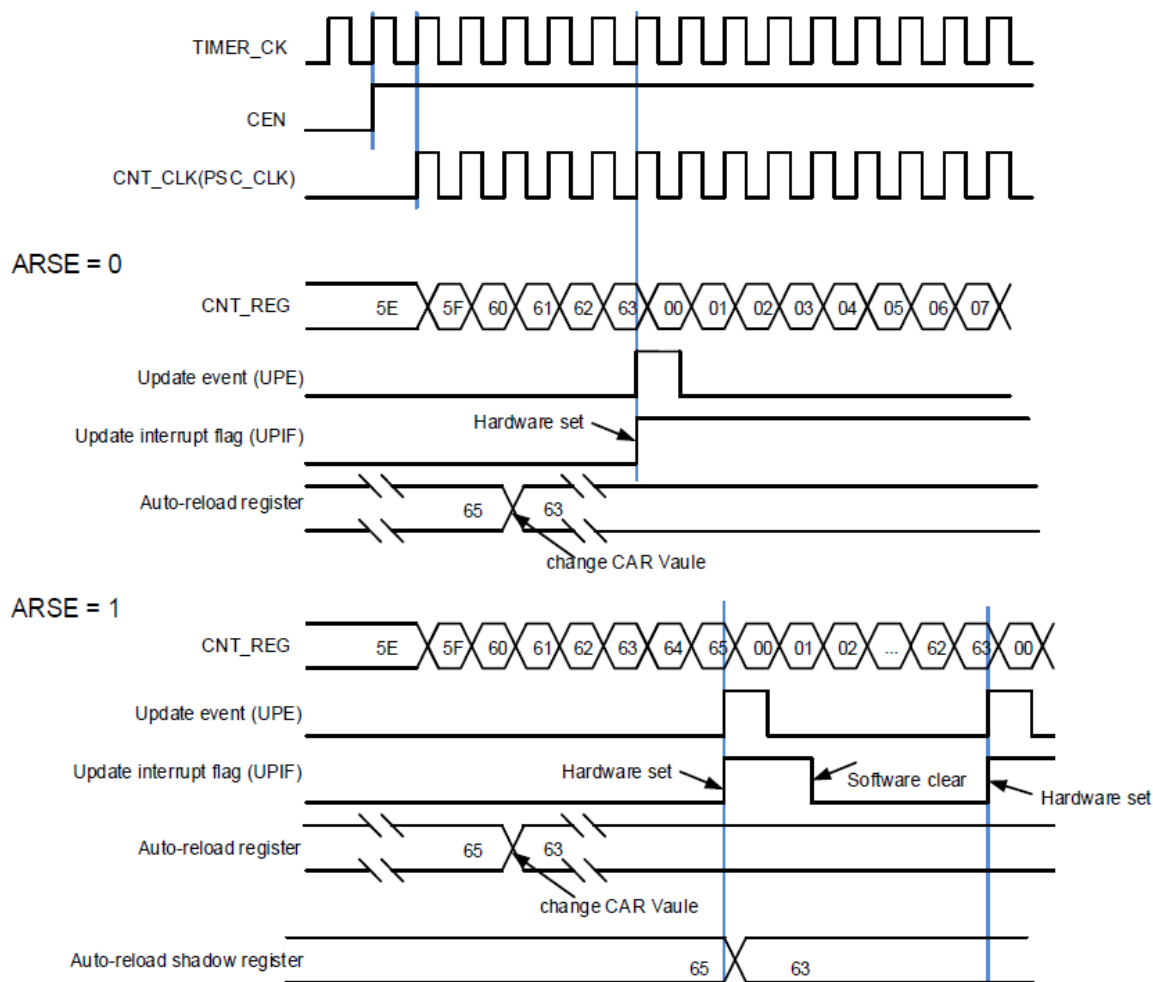


Рис.6.2.2

Диаграмма работы таймера при счёте вниз представлена на рис.6.2.3.

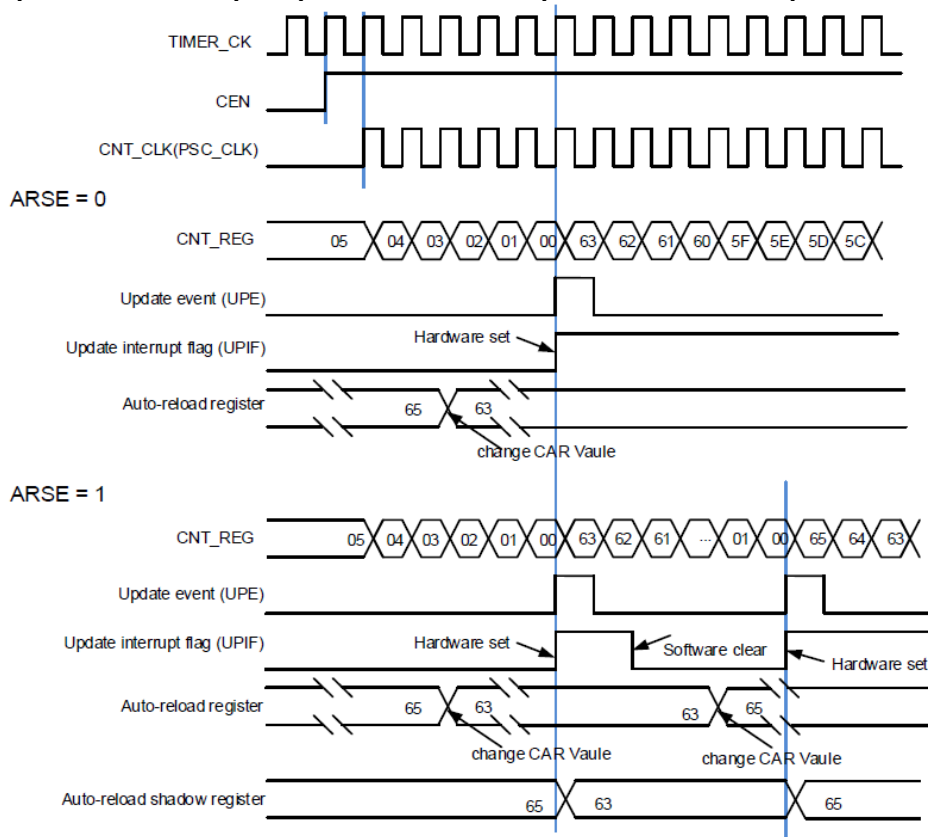


Рис.6.2.3

Диаграмма работы таймера при счёте вверх и вниз представлена на рис.6.2.4.

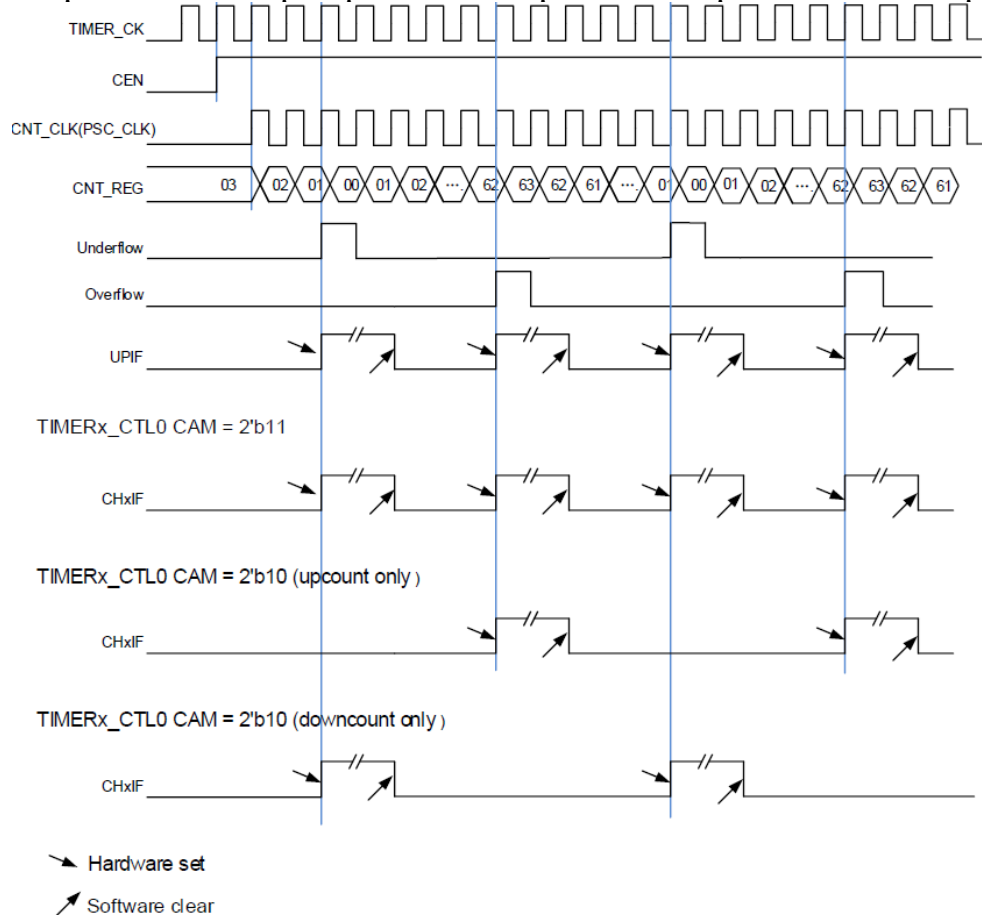


Рис.6.2.4

Для настройки канала таймера на формирование широтно-модулированного сигнала необходимо объявить структуру типа `timer_oc_parameter_struct`.

```
typedef struct
{
    uint16_t outputstate;           /*!< channel output state */
    uint16_t outputnstate;         /*!< channel complementary output state */
    uint16_t ocpolarity;           /*!< channel output polarity */
    uint16_t ocnpolarity;          /*!< channel complementary output polarity */
    /*
    uint16_t ocidlestate;          /*!< idle state of channel output */
    uint16_t ocnidlestate;         /*!< idle state of channel complementary
output */
}timer_oc_parameter_struct;
```

В поле `outputstate` записывается параметр разрешения прямого выхода канала:

`TIMER_CCX_ENABLE` – канал разрешён;

`TIMER_CCX_DISABLE` – канал запрещён.

В поле `outputnstate` записывается параметр разрешения инверсного выхода канала:

`TIMER_CCXN_ENABLE` – канал разрешён;

`TIMER_CCXN_DISABLE` – канал запрещён.

В поле `ocpolarity` записывается параметр полярности прямого выхода канала:

`TIMER_OC_POLARITY_HIGH` – активный уровень сигнала 1;

TIMER_OC_POLARITY_LOW – активный уровень сигнала 0.

В поле ocnpolarity записывается параметр полярности инверсного выхода канала:

TIMER_OCN_POLARITY_HIGH – активный уровень сигнала 1;

TIMER_OCN_POLARITY_LOW – активный уровень сигнала 0.

В поле ocidlestate записывается параметр полярности прямого выхода канала в выключенном состоянии:

TIMER_OC_IDLE_STATE_HIGH – уровень сигнала 1;

TIMER_OC_IDLE_STATE_LOW – уровень сигнала 0.

В поле ocidlestate записывается параметр полярности прямого выхода канала в выключенном состоянии:

TIMER_OCN_IDLE_STATE_HIGH – уровень сигнала 1;

TIMER_OCN_IDLE_STATE_LOW – уровень сигнала 0.

Инициализация канала таймера на формирование широтно-модулированного сигнала осуществляется функцией

```
void timer_channel_output_config(uint32_t timer_periph, uint16_t channel, timer_oc_parameter_struct* ocpara)
```

где:

timer_periph – номер таймера TIMER0 – TIMER13;

channel – номер канала TIMER_CH_0 - TIMER_CH_3;

ocpara – указатель на структуру инициализации канала.

Пример инициализации канала приведён ниже.

```
/* CH0/CH0N configuration in PWM mode0 */
```

```
timer_ocintpara.outputstate = TIMER_CCX_ENABLE;
```

```
// Timer CH channel enable
```

```
timer_ocintpara.outputnstate = TIMER_CCXN_DISABLE;
```

```
// Timer CHN channel enable
```

```
timer_ocintpara.ocpolarity = TIMER_OC_POLARITY_HIGH;
```

```
// CH Polarity high
```

```
timer_ocintpara.ocnpolarity = TIMER_OCN_POLARITY_LOW;
```

```
// CHN Polarity high
```

```
timer_ocintpara.ocidlestate = TIMER_OC_IDLE_STATE_HIGH;
```

```
// CH Idle state low
```

```
timer_ocintpara.ocnidlestate = TIMER_OCN_IDLE_STATE_HIGH;
```

```
// CHN Idle state low
```

```
timer_channel_output_config(TIMER0, TIMER_CH_0, &timer_ocintpara);
```

В этом режиме текущее значение таймера сравнивается с содержимым регистра TIMER_CHXCV. При равенстве этих значений происходит изменение выходного сигнала канала таймера. Это вызывает запрос прерывания канала таймера TIMER_FLAG_CHX. Этот запрос прерывания должен быть сброшен программно с помощью функции

```
timer_interrupt_flag_clear(TIMEX, TIMER_INT_CHX).
```

Диаграмма работы канала таймера в режиме ШИМ при счёте вверх, вниз и туда и обратно представлена на рис.6.2.5 и рис.6.2.6.

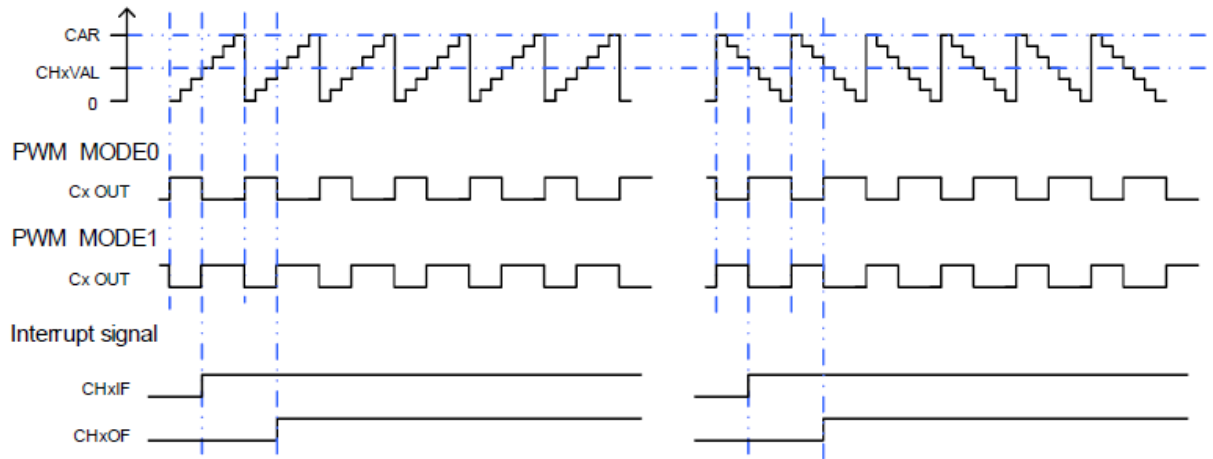


Рис.6.2.5

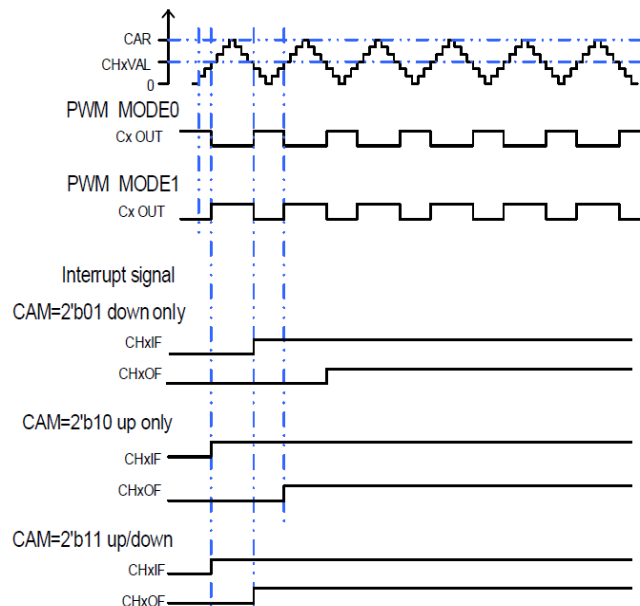


Рис.6.2.6

Включение - выключение выходной функции канала таймера осуществляется с помощью функции

```
void timer_primary_output_config(uint32_t timer_periph, ControlStatus newvalue),
```

где:

timer_periph – номер таймера TIMER0 – TIMER13;

ControlStatus – ENABLE, DISABLE.

Включение таймера осуществляется с помощью функции

```
void timer_enable(uint32_t timer_periph),
```

где:

timer_periph – номер таймера TIMER0 – TIMER13.

Пример инициализация канала таймера на формирование широтно-модулированного сигнала приведён ниже.

```
/* CH0/CH0N configuration in PWM mode0 */
timer_ocintpara.outputstate = TIMER_CCX_ENABLE;
    // Timer CH channel enable
timer_ocintpara.outputnstate = TIMER_CCXN_DISABLE;
    // Timer CHN channel enable
timer_ocintpara.ocpolarity = TIMER_OC_POLARITY_HIGH;
// CH Polarity high
timer_ocintpara.ocnpolarity = TIMER_OCN_POLARITY_LOW;
// CHN Polarity high
timer_ocintpara.ocidlestate = TIMER_OC_IDLE_STATE_HIGH;
// CH Idle state low
timer_ocintpara.ocnidlestate = TIMER_OCN_IDLE_STATE_HIGH;
// CHN Idle state low

timer_channel_output_config(TIMER0, TIMER_CH_0, &timer_ocintpara);
```

Запуск канала таймера на формирование широтно-модулированного сигнала осуществляется функцией

```
void timer_channel_output_mode_config(uint32_t timer_periph, uint16_t channel,
uint16_t ocmode)
```

где:

timer_periph – номер таймера TIMER0 – TIMER13;

channel – номер канала TIMER_CH_0 - TIMER_CH_3;

ocmode – тип режима ШИМ:

TIMER_OC_MODE_PWM0 – режим ШИМ0;

TIMER_OC_MODE_PWM1 - режим ШИМ1.

Включение зеркальных регистров канала таймера осуществляется функцией

```
void timer_channel_output_shadow_config(uint32_t timer_periph, uint16_t channel,
uint16_t ocshadow)
```

где:

timer_periph – номер таймера TIMER0 – TIMER13;

channel – номер канала TIMER_CH_0 - TIMER_CH_3;

ocshadow – параметр включения заркальных регистров:

TIMER_OC_SHADOW_ENABLE – зеркальные регистры включены;

TIMER_OC_SHADOW_DISABLE – зеркальные регистры выключены.

Включение зеркальных регистров необходимо для предотвращения множественных переключений выхода канала таймера на периоде ШИМ,

Запись нового значения длительности импульса ШИМ в канал таймера таймера осуществляется функцией

```
void timer_channel_output_pulse_value_config(uint32_t timer_periph, uint16_t channel,
uint32_t pulse)
```

где:

Для настройки канала таймера на выполнение входной функции захвата необходимо объявить структуру типа `timer_ic_parameter_struct`.

```
typedef struct
{
    uint16_t icpolarity;           /*!< channel input polarity */
    uint16_t icselection;          /*!< channel input mode selection */
    uint16_t icprescaler;          /*!< channel input capture prescaler */
    uint16_t icfilter;             /*!< channel input capture filter control */
}timer_ic_parameter_struct;
```

В поле `icpolarity` записывается параметр типа фронта фиксации:

`TIMER_IC_POLARITY_RISING` – фиксируется перепад 0 в 1;

`TIMER_IC_POLARITY_FALLING` – фиксируется перепад 1 в 0.

В поле `icselection` записывается параметр входа фиксации. Он должен быть установлен в значение `TIMER_IC_SELECTION_DIRECTTI`, что означает фиксацию входа данного канала.

В поле `icprescaler` записывается параметр прореживания фиксации фиксации:

`TIMER_IC_PSC_DIV1` – фиксируется каждый перепад;

`TIMER_IC_PSC_DIV2` – фиксируется каждый второй перепад;

`TIMER_IC_PSC_DIV4` – фиксируется каждый четвёртый перепад;

`TIMER_IC_PSC_DIV8` – фиксируется каждый восьмой перепад.

В поле `icfilter` записывается параметр, определяющий режим работы цифрового фильтра фиксации:

0000: Filter disabled, $fSAMP=fDTS$, $N=1$

0001: $fSAMP=fTIMER_CK$, $N=2$

0010: $fSAMP=fTIMER_CK$, $N=4$

0011: $fSAMP=fTIMER_CK$, $N=8$

0100: $fSAMP=fDTS/2$, $N=6$

0101: $fSAMP=fDTS/2$, $N=8$

0110: $fSAMP=fDTS/4$, $N=6$

0111: $fSAMP=fDTS/4$, $N=8$

1000: $fSAMP=fDTS/8$, $N=6$

1001: $fSAMP=fDTS/8$, $N=8$

1010: $fSAMP=fDTS/16$, $N=5$

1011: $fSAMP=fDTS/16$, $N=6$

1100: $fSAMP=fDTS/16$, $N=8$

1101: $fSAMP=fDTS/32$, $N=5$

1110: $fSAMP=fDTS/32$, $N=6$

1111: $fSAMP=fDTS/32$, $N=8$

Инициализация канала таймера на выполнение входной функции захвата осуществляется функцией

```
void timer_input_capture_config(uint32_t timer_periph,uint16_t channel, timer_ic_parameter_struct* icpara)
```

где:

`timer_periph` – номер таймера `TIMER0` – `TIMER13`;

`channel` – номер канала `TIMER_CH_0` - `TIMER_CH_3`;

`icpara` – имя структуры инициализации.

Пример инициализация канала таймера на выполнение входной функции захвата приведён ниже.

```
/* TIMER0 CH2 input capture configuration */
timer_icinitpara.icpolarity = TIMER_IC_POLARITY_RISING;
timer_icinitpara.icselection = TIMER_IC_SELECTION_DIRECTTI;
timer_icinitpara.icprescaler = TIMER_IC_PSC_DIV1;
timer_icinitpara.icfilter = 0x0;
timer_input_capture_config(TIMER0, TIMER_CH_2, &timer_icinitpara);
```

Запись нового значения счётчика таймера осуществляется с помощью функции

```
void timer_counter_value_config(uint32_t timer_periph, uint16_t counter),
```

где:
timer_periph – номер таймера TIMER0 – TIMER13;
counter – новое значение счётчика.

Разрешение обработки прерывания таймера осуществляется с помощью функции

```
void timer_interrupt_enable(uint32_t timer_periph, uint32_t interrupt),
```

где:
timer_periph – номер таймера TIMER0 – TIMER13;
interrupt – наименование конкретного вектора прерывания таймера:
TIMER_INT_UP – прерывание события перезагрузки таймера;
TIMER_INT_CH0 - TIMER_INT_CH3 – прерывания канала таймера;
TIMER_INT_CMT – прерывание коммутации таймера;
TIMER_INT_TRG – прерывание события триггера таймера;
TIMER_INT_BRK – прерывание защитного входа канала таймера.

Сброс флага прерывания таймера осуществляется с помощью функции

```
void timer_interrupt_flag_clear(uint32_t timer_periph, uint32_t interrupt)
```

где:
timer_periph – номер таймера TIMER0 – TIMER13;
interrupt – тип флага прерывания таймера:
TIMER_INT_FLAG_UP – флаг прерывания перезагрузки таймера;
TIMER_INT_FLAG_CH0 - флаг прерывания канала 0 таймера;
TIMER_INT_FLAG_CH1 - флаг прерывания канала 1 таймера;
TIMER_INT_FLAG_CH2 - флаг прерывания канала 2 таймера;
TIMER_INT_FLAG_CH3 - флаг прерывания канала 3 таймера.

Для того, чтобы таймеры функционировали необходимо предварительно включить их синхронизацию. Для включения синхронизации портов ввода/вывода используется функция

```
void rcu_periph_clock_enable(rcu_periph_enum periph), где  
periph - RCU_TIMERx (x=0 - 13).
```

6.3. Интерфейс SPI

МК GD32F303CC имеет в своём составе 3 модуля SPI. Модуль SPI обеспечивает обмен данными по полнодуплексному синхронному последовательному интерфейсу. SPI может выполнять функции ведущего или ведомого устройства в дуплексном или полудуплексном режимах, а также поддерживает работу нескольких ведомых и ведущих устройств на одной шине. Сигнал выбора ведомого (NSS) можно настроить как вход выбора SPI в ведомом режиме или как выход выбора кристалла в ведущем режиме. В ведущем режиме можно также использовать порты ввода/вывода общего назначения для выбора нескольких ведомых устройств. Модуль SPI0 тактируется сигналом синхронизации периферийной шины APB1, а модули SPI1, SPI2 - сигналом синхронизации периферийной шины APB1.

Структура модуля SPI представлена на рис.6.3.1.

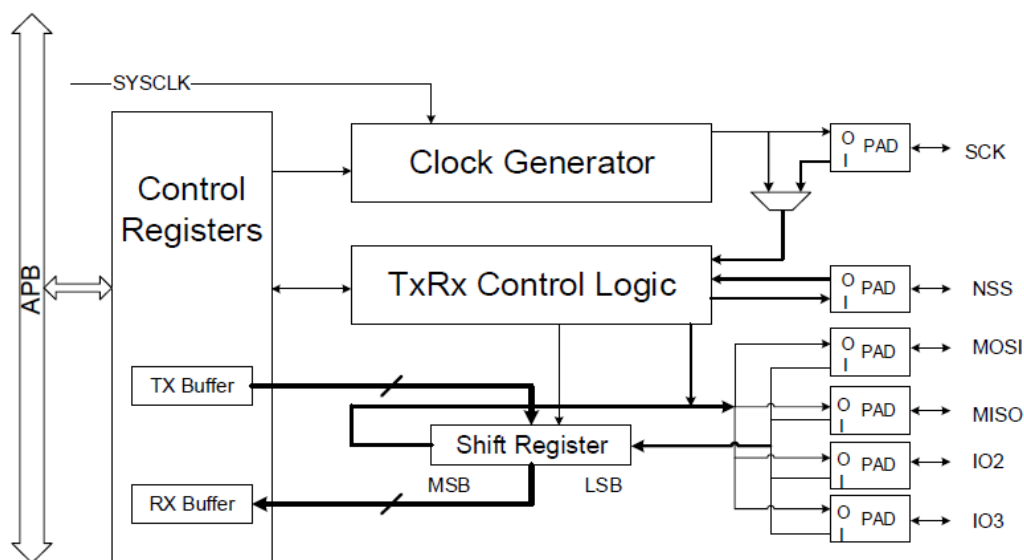


Рис. 6.3.1 Структурная схема модуля SPI

Модуль SPI0 синхронизируется от внутренней периферийной шины APB2, а модули SPI1 и SPI2 – от шины APB1.

6.3.1. Описание сигналов

Сигнал MOSI (master-out, slave-in - «выход ведущего, вход ведомого») является выходом данных ведущего устройства и входом данных ведомых устройств. Он используется для последовательной передачи данных от ведущего к ведомому. Этот сигнал является выходом, если модуль SPI работает в ведущем режиме, и входом, если модуль SPI работает в ведомом режиме. Данные могут передаваться старшими или младшими значащими разрядами вперед.

Сигнал MISO (master-in, slave-out - «вход ведущего, выход ведомого») является выходом данных ведомого устройства и входом данных ведущего устройства. Он используется для последовательной передачи данных от ведомого к ведущему. Этот сигнал является входом, если модуль SPI работает в ведущем ре-

жиме, и выходом, если модуль SPI работает в ведомом режиме. Данные могут передаваться старшими или младшими значащими разрядами вперед. Вывод MISO переводится в высокоимпедансное состояние, когда модуль SPI отключен, а также тогда, когда модуль SPI работает в как ведомый, который не выбран.

Сигнал SCK (serial clock – «импульсы тактирования последовательного интерфейса») является выходом ведущего устройства и входом ведомых устройств. Он используется для синхронизации обмена данными между ведущим и ведомым устройствами по линиям MOSI и MISO. Модуль SPI генерирует этот сигнал, когда работает в ведущем режиме. Сигнал SCK игнорируется ведомым SPI, когда ведомый не выбран ($NSS = 1$).

Сигнал выбора ведомого NSS (slave select) является сигналом выбора активного ведомого устройства на шине SPI. Этот сигнал является выходом, если модуль SPI работает в ведущем режиме, и входом, если модуль SPI работает в ведомом режиме. Этот сигнал может быть настроен на аппаратное или программное формирование. При аппаратном формировании активный нулевой уровень сигнала устанавливается и снимается аппаратно соответственно в начале и в конце передачи пакета данных. При программном формировании этот сигнал должен в случае необходимости изменяться программно с помощью функций работы с портами ввода - вывода.

На рисунке 6.3.2 показана типичная схемы включения ведущего и ведомого устройств на шине SPI в дуплексном режиме, а на рисунке 6.3.3 - в полудуплексном режиме

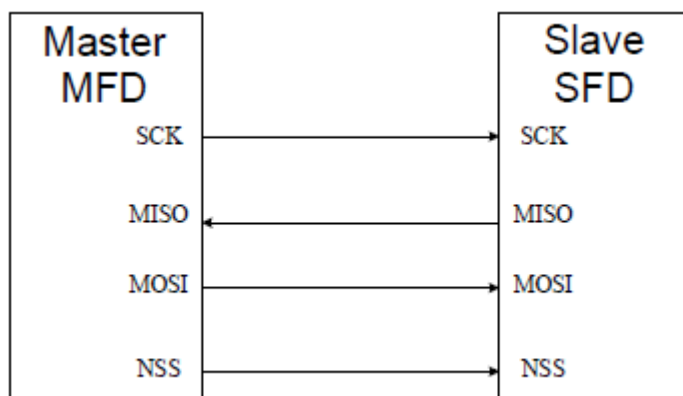


Рис. 6.3.2

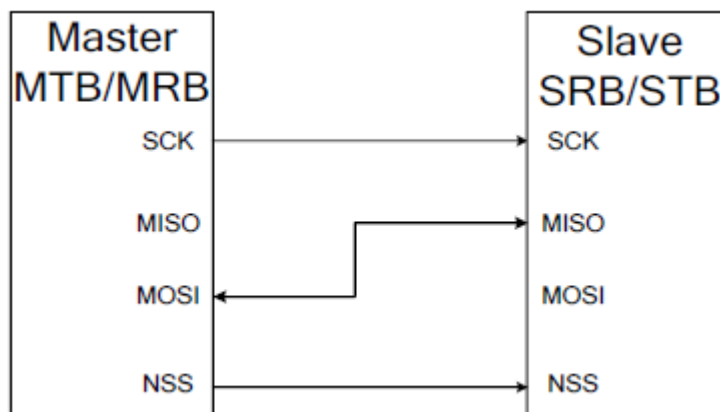


Рис. 6.3.3

На рис.6.3.4 представлена диаграмма передачи пакета данных при разных вариантах настройки синхронизации

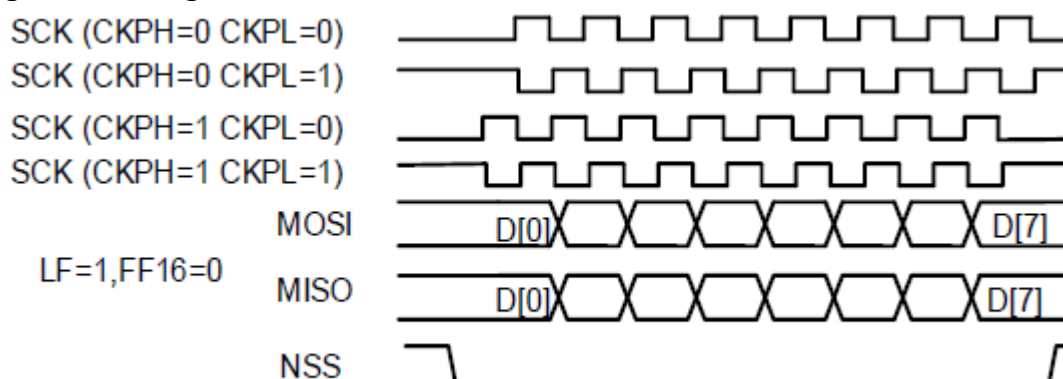


Рис. 6.3.4

Для инициализации модуля SPI в программе необходимо объявить структуру типа `spi_parameter_struct`.

```
typedef struct
{
    uint32_t device_mode;           /*!< SPI master or slave */
    uint32_t trans_mode;           /*!< SPI transtype */
    uint32_t frame_size;           /*!< SPI frame size */
    uint32_t nss;                  /*!< SPI NSS control by hardware or software */
    uint32_t endian;               /*!< SPI big endian or little endian */
    uint32_t clock_polarity_phase; /*!< SPI clock phase and polarity */
    uint32_t prescale;             /*!< SPI prescale factor */
} spi_parameter_struct;
```

В поле `device_mode` записывается режим работы модуля:

SPI_MASTER – режим ведущего;

SPI_SLAVE – режим ведомого.

В поле `trans_mode` записывается режим обмена информацией:

SPI_TRANSMODE_FULLDUPLEX – полнодуплексный режим;

SPI_TRANSMODE_RECEIVEONLY – только приём информации;

SPI_TRANSMODE_BDRECEIVE – двунаправленный приём по одной линии;

SPI_TRANSMODE_BDTRANSMIT – двунаправленная передача по одной линии.

В поле `frame_size` записывается длина пакета данных:

SPI_FRAME_SIZE_16BIT – 16 бит;

SPI_FRAME_SIZE_8BIT – 8 бит.

В поле `nss` записывается режим контроля вывода NSS:

SPI_NSS_SOFT – программный контроль;

SPI_NSS_HARD – аппаратный контроль.

В поле `endian` записывается очередность выдачи битов данных:

SPI_ENDIAN_MSB – старшими битами вперёд;

SPI_ENDIAN_LSB – младшими битами вперёд.

В поле clock_polarity_phase записывается полярность сигнала SCK и фазировка данных относительно него (рис. 6.3.4):

SPI_CK_PL_LOW_PH_1EDGE – SCK в исходном состоянии 0, данные выставляются по отрицательному перепаду, фиксируются по положительному перепаду;

SPI_CK_PL_HIGH_PH_1EDGE – SCK в исходном состоянии 1, данные выставляются по положительному перепаду, фиксируются по отрицательному перепаду;

SPI_CK_PL_LOW_PH_2EDGE – SCK в исходном состоянии 0, данные выставляются по положительному перепаду, фиксируются по отрицательному перепаду;

SPI_CK_PL_HIGH_PH_2EDGE – SCK в исходном состоянии 1, данные выставляются по отрицательному перепаду, фиксируются по положительному перепаду.

В поле prescale записывается делитель частоты системной шины для определения частоты сигнала SCK:

SPI_PSC_2 – коэффициент деления 2;

SPI_PSC_4 – коэффициент деления 4;

SPI_PSC_8 – коэффициент деления 8;

SPI_PSC_16 – коэффициент деления 16;

SPI_PSC_32 – коэффициент деления 32;

SPI_PSC_64 – коэффициент деления 64;

SPI_PSC_128 – коэффициент деления 128;

SPI_PSC_256 – коэффициент деления 256.

Инициализация модуля SPI осуществляется с помощью функции void spi_init(uint32_t spi_periph, spi_parameter_struct* spi_struct), где spi_periph – номер таймера SPI0 – SPI2;

spi_struct – указатель на структуру инициализации модуля SPI.

Пример инициализации модуля SPI приведён ниже.

```
spi_parameter_struct spi_init_struct;
```

```
rcu_periph_clock_enable(RCU_SPI2);
```

```
// For PA15 remaping
```

```
rcu_periph_clock_enable(RCU_AF);
```

```
// PA15 - GPIO
```

```
gpio_pin_remap_config(GPIO_SWJ_SWDPENABLE_REMAP,    ENA-  
BLE);
```

```
/* SPI0 GPIO config:SCK/PA5, MISO/PA6, MOSI/PA7 */
```

```
gpio_init(GPIOB,      GPIO_MODE_AF_OD,      GPIO_OSPEED_50MHZ,  
GPIO_PIN_3);
```

```
gpio_init(GPIOB,  GPIO_MODE_IN_FLOATING,  GPIO_OSPEED_50MHZ,  
GPIO_PIN_4);
```

```

/* PA3 as NSS */
gpio_init(GPIOA,      GPIO_MODE_OUT_OD,      GPIO_OSPEED_50MHZ,
GPIO_PIN_15);
//      gpio_init(GPIOA,  GPIO_MODE_AF_PP,  GPIO_OSPEED_50MHZ,
GPIO_PIN_15);

/* SPI0 parameter config */
spi_init_struct.trans_mode      = SPI_TRANSMODE_FULLDUPLEX;
spi_init_struct.device_mode     = SPI_MASTER;
spi_init_struct.frame_size      = SPI_FRAME_SIZE_8BIT;
spi_init_struct.clock_polarity_phase = SPI_CLK_PL_HIGH_PH_2EDGE;
spi_init_struct.nss              = SPI_NSS_SOFT;
spi_init_struct.prescale         = SPI_PSC_256;
spi_init_struct.endian           = SPI_ENDIAN_MSB;
spi_init(SPI2, &spi_init_struct);

spi_enable(SPI2);

```

Глобальное разрешение работы модуля SPI осуществляется функцией

`void spi_enable(uint32_t spi_periph)` где:

`spi_periph` – номер таймера SPI0 – SPI2.

Для того, чтобы модули SPI функционировали необходимо предварительно включить их синхронизацию. Для включения синхронизации портов ввода/вывода используется функция

`void rcu_periph_clock_enable(rcu_periph_enum periph)`, где

`periph` - RCU_SPIx (x=0 - 2).

Активизация обмена данными через модуль SPI в режиме ведущего осуществляется с помощью функции

`void spi_i2s_data_transmit(uint32_t spi_periph, uint16_t data)`, где:

`spi_periph` – номер таймера SPI0 – SPI2;

`data` – передаваемые данные.

Чтение флагов состояния модуля SPI осуществляется с помощью функции

`void spi_i2s_data_transmit(uint32_t spi_periph, uint16_t data)` где:

`spi_periph` – номер таймера SPI0 – SPI2;

`FlagStatus spi_i2s_flag_get(uint32_t spi_periph, uint32_t flag)`, где:

`spi_periph` – номер таймера SPI0 – SPI2;

`flag` – имя флага:

SPI_FLAG_TBE: transmit buffer empty flag

SPI_FLAG_RBNE: receive buffer not empty flag

SPI_FLAG_TRANS: transmit on-going flag

SPI_FLAG_RXORERR: receive overrun error flag

SPI_FLAG_CONFERR: mode config

error flag

SPI_FLAG_CRCERR: CRC error flag

SPI_FLAG_FERR: format error flag.

Функция возвращает значения:

RESET – флаг не установлен;

SET – флаг установлен.

Чтение принятых данных осуществляется функцией `uint16_t spi_i2s_data_receive(uint32_t spi_periph)`, где `spi_periph` – номер таймера SPI0 – SPI2.

Для того, чтобы модуль SPI функционировали необходимо предварительно включить их синхронизацию. Для включения синхронизации портов ввода/вывода используется функция

`void rcu_periph_clock_enable(rcu_periph_enum periph)`, где `periph` - RCU_SPI2x (x=0 - 2).

6.4. Интерфейс UART

МК GD32F303CC имеет в своём составе 5 модулей UART. UART представляет собой асинхронный полнодуплексный последовательный порт. Он, в частности, может быть использован для связи с компьютером через преобразователь UART – USB.

Модуль UART0 тактируется сигналом синхронизации периферийной шины APB1, а модули UART1 - UART4 - сигналом синхронизации периферийной шины APB1.

Структура модуля UART представлена на рис.6.4.1.

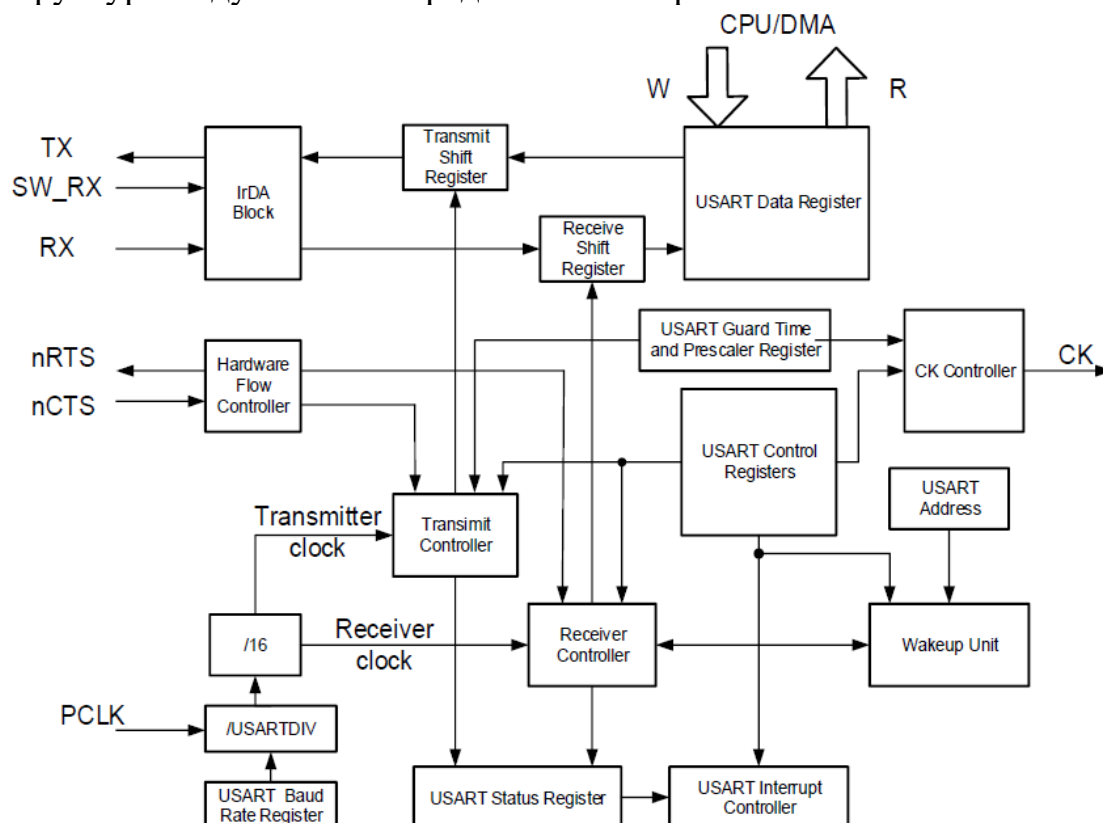


Рис. 6.4.1

6.4.1 Описание сигналов

Сигнал TX (Transmit Data) является выходом данных передатчика UART. При отсутствии передачи сигнал находится в единичном состоянии. Во время передачи на вывод TX последовательно выдвигаются биты посылки младшими битами вперёд с заданной частотой.

Сигнал RX (Receive Data) является входом данных приёмника UART. При отсутствии передачи сигнал должен находиться в единичном состоянии. Перепад сигнала из единичного в нулевое состояние («СТАРТ - бит») является для приёмника признаком начала передачи. Передача должна завершаться перепадом сигнала из нулевого в единичное состояние («СТОП - бит»).

nCTS (Clear to Send) является входным сигналом готовности к приёму данных от передатчика UART. Он используется при аппаратном контроле потоком данных. Передатчик постоянно отслеживает состояние своей линии nCTS и, увидев пассивный (единичный) уровень сигнала, аппаратно прекращает отправку данных.

nRTS (Request to Send) является выходным сигналом запроса передачи данных от передатчика UART. Он используется при аппаратном контроле потоком данных. Когда приёмник освобождает свой входной буфер, он выставляет активный (нулевой) уровень сигнала nRTS, сигнализируя, что готов к приёму данных. Передатчик видит это на своей линии nCTS и возобновляет передачу.

Длина пакета данных может быть 8 или 9 бит. На рисунке 6.4.2 показана структура пакета данных UART длиной 8 бит.

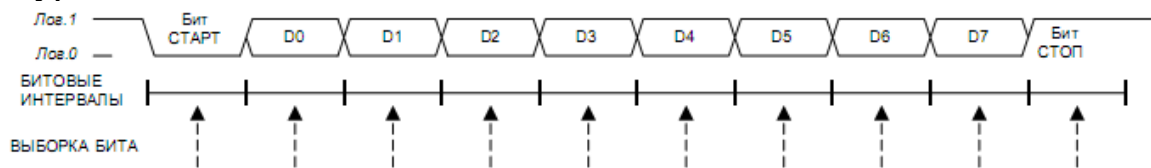


Рис. 6.4.2

На рисунке 6.4.3 показана структура пакета данных UART длиной 9 бит.

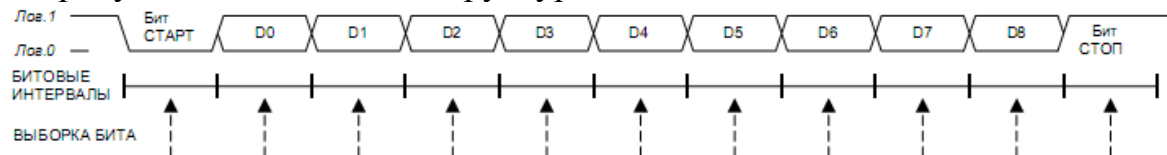
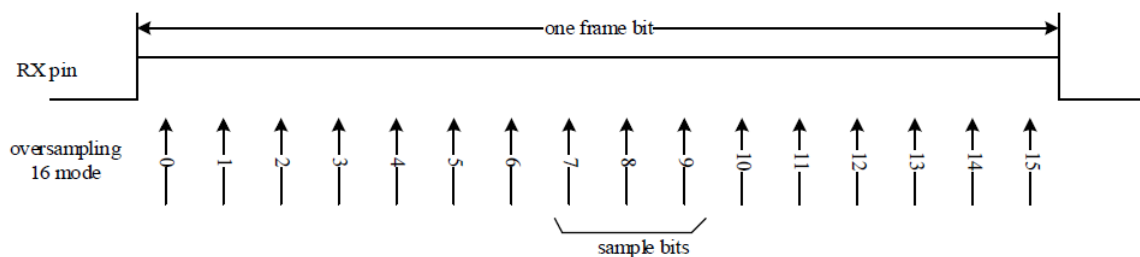


Рис. 6.4.3

В качестве девятого бита может быть использован бит чётности.

Входные данные стробируются с передискретизацией в 16 раз относительно заданной частоты передачи для уменьшения рассинхронизации приёмника и передатчика (рис.6.4.4). Седьмой, восьмой и девятый отсчёты выборки используются для мажоритарного выбора правильного значения сигнала.



типичная схемы включения ведущего и ведомого устройств на шине SPI в дуплексном режиме, а на рисунке 6.3.3 - в полудуплексном режиме

Для инициализации модуля UART необходимо задать ряд параметров его работы, каждый из которых задаётся отдельной процедурой.

Частота смены бит в посылке задаётся с помощью процедуры `void usart_baudrate_set(uint32_t usart_periph, uint32_t baudval)`, где `usart_periph` – модуля UART USARTx(x=0,1,2); `baudval` – значение частоты смены бит в посылке в битах/секунду.

Наличие и тип бита чётности в послыке задаётся с помощью процедуры `void usart_parity_config(uint32_t usart_periph, uint32_t paritycfg)`, где `usart_periph` – модуля UART USARTx(x=0,1,2); `paritycfg` – параметр, определяющий наличие и тип бита чётности: `USART_PM_NONE` – нет бита чётности; `USART_PM_ODD` – бит нечётности; `USART_PM_EVEN` – бит чётности.

Длина послыки в битах задаётся с помощью процедуры `void usart_word_length_set(uint32_t usart_periph, uint32_t wlen)`, где `usart_periph` – модуля UART USARTx(x=0,1,2); `wlen` – параметр, определяющий длину послыки в битах: `USART_WL_8BIT` – 8 бит; `USART_WL_9BIT` – 9 бит.

Размер СТОП-бита в послыке задаётся с помощью процедуры `void usart_stop_bit_set(uint32_t usart_periph, uint32_t stblen)`, где `usart_periph` – модуля UART USARTx(x=0,1,2); `stblen` – параметр, определяющий размер СТОП-бита в послыке: `USART_STB_1BIT` – 1 СТОП-бит; `USART_STB_0_5BIT` – 0.5 СТОП-бита; `USART_STB_2BIT` – 2 СТОП-бита; `USART_STB_1_5BIT` – 1.5 СТОП-бита.

Разрешение приёма послылок задаётся с помощью процедуры `void usart_receive_config(uint32_t usart_periph, uint32_t rxconfig)`, где `usart_periph` – модуля UART USARTx(x=0,1,2); `rxconfig` – параметр, определяющий разрешение приёма послылок: `USART_RECEIVE_ENABLE` – приём послылок разрешён; `USART_RECEIVE_DISABLE` – приём послылок запрещён.

Разрешение передачи послылок задаётся с помощью процедуры `void usart_transmit_config(uint32_t usart_periph, uint32_t txconfig)`, где `usart_periph` – модуля UART USARTx(x=0,1,2); `txconfig` – параметр, определяющий разрешение передачи послылок: `USART_TRANSMIT_ENABLE` – передача послылок разрешена; `USART_TRANSMIT_DISABLE` – передача послылок запрещена.

Глобальное разрешение работы модуля UART задаётся с помощью процедуры

`void usart_enable(uint32_t usart_periph)`, где `usart_periph` – модуля UART USARTx(x=0,1,2).

Пример инициализации модуля UART приведён ниже.

```
rcu_periph_clock_enable(RCU_USART2);
```

```
// Configure PB10 as USART2_TX
gpio_init(GPIOB, GPIO_MODE_AF_PP, GPIO_OSPEED_50MHZ,
GPIO_PIN_10);
```

```
// Configure PB11 as USART2_RX
```

```
    gpio_init(GPIOB, GPIO_MODE_IN_FLOATING, GPIO_OSPEED_50MHZ,  
GPIO_PIN_11);
```

```
// USART configure  
    usart_deinit(USART2);  
    usart_baudrate_set(USART2, 9600U);  
    usart_parity_config(USART2, USART_PM_NONE);  
    usart_word_length_set(USART2, USART_WL_8BIT);  
    usart_stop_bit_set(USART2, USART_STB_1BIT);  
    usart_receive_config(USART2, USART_RECEIVE_ENABLE);  
    usart_transmit_config(USART2, USART_TRANSMIT_ENABLE);  
    usart_enable(USART2);
```

Функция `usart_deinit` осуществляет начальную деинициализацию модуля UART.

Инициация передачи данных через модуль UART осуществляется с помощью процедуры

```
void usart_data_transmit(uint32_t usart_periph, uint32_t data), где  
usart_periph – модуля UART USARTx(x=0,1,2);  
data – передаваемые данные.
```

Чтение полученных через модуль UART данных осуществляется с помощью процедуры

```
FlagStatus usart_flag_get(uint32_t usart_periph, usart_flag_enum flag), где  
usart_periph – модуля UART USARTx(x=0,1,2).
```

Чтение флагов состояния модуля UART осуществляется с помощью процедуры

```
void usart_data_transmit(uint32_t usart_periph, uint32_t data), где  
usart_periph – модуля UART USARTx(x=0,1,2);  
flag – тип флага состояния:  
USART_FLAG_RBNE – в буфере приёмника есть непрочитанные данные;  
USART_FLAG_TBE – буфер передатчика пуст;  
USART_FLAG_ORERR – ошибка перезаписи в буфере приёмника;  
USART_FLAG_PERR – ошибка чётности в принятой посылке;  
USART_FLAG_FERR – ошибка формата посылки;  
USART_FLAG_BSY – флаг занятости приёмника.
```

Функция возвращает значения:

RESET – флаг не установлен;

SET – флаг установлен.

Пример процедур передачи и приёма пакетов данных через модуль UART приведён ниже.

```
void UART2_Transmit(uint16_t Data)  
{  
    while(RESET == usart_flag_get(USART2, USART_FLAG_TBE)){}  
  
    // write one byte in the USART1 data register  
    usart_data_transmit(USART1, Data);
```

```
// wait until end of transmit
// while(RESET == usart_flag_get(UART3, USART_FLAG_TC)){}
}

uint16_t UART2_Receive(void)
{
    while(RESET == usart_flag_get(USART2, USART_FLAG_RBNE)){}

    return usart_data_receive(USART2);
}
```

6.5. Аналогово-цифровой преобразователь

МК GD32F303CC имеет в своём составе 3 аналого-цифровых преобразователя (АЦП). Каждый из них имеет 10 входных аналоговых каналов. Разрядность каждого АЦП может быть программно выбрана между значениями 6, 8, 10 или 12 бит.

Структурная схема модуля АЦП представлена на рис.6.5.1.

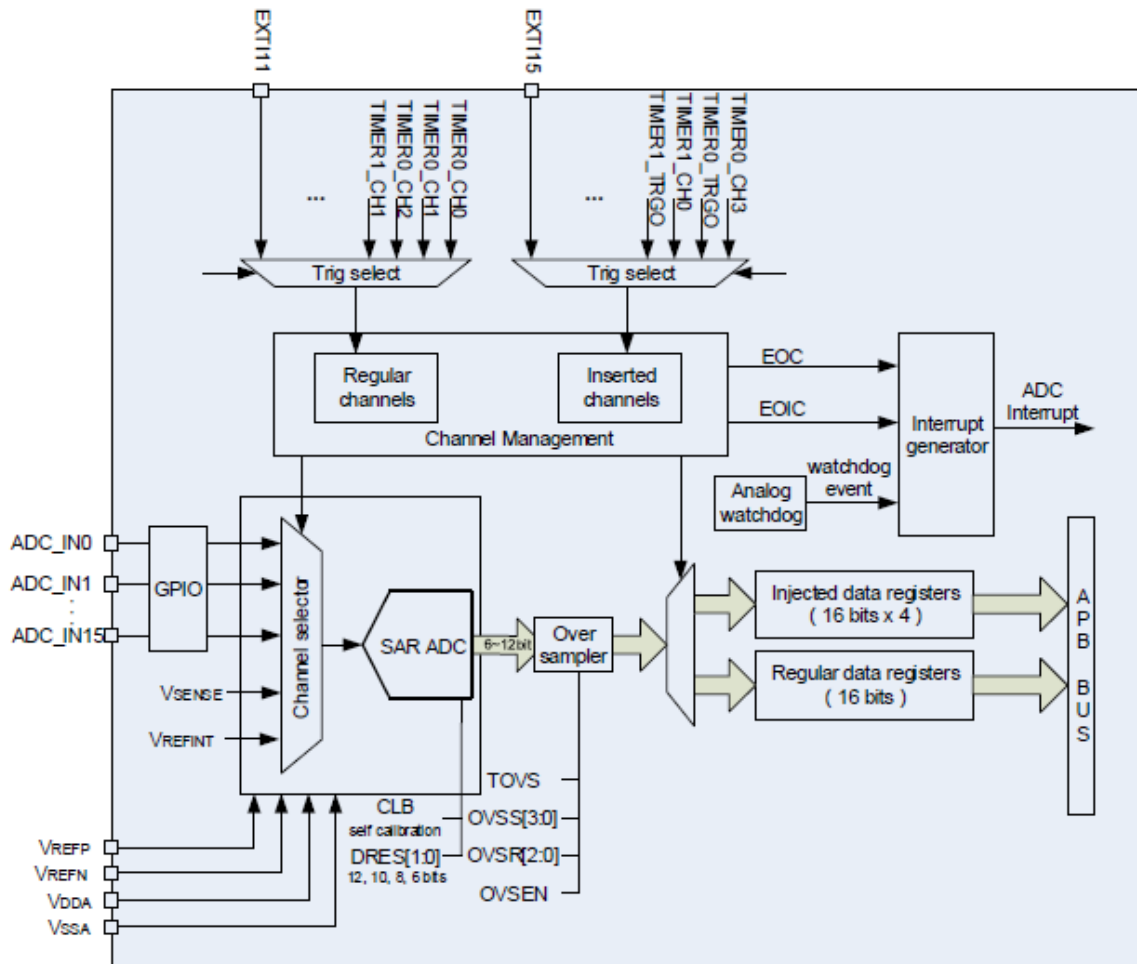


Рис.6.5.1

Модули АЦП тактируются сигналом синхронизации периферийной шины APB2 через программируемый делитель частоты. Коэффициент деления частоты задаётся с помощью функции

```
void rcu_adc_clock_config(uint32_t adc_psc),
```

где `adc_psc` – коэффициент деления частоты:

`RCU_CKADC_CKAPB2_DIV2`: ADC prescaler select `CK_APB2/2`

`RCU_CKADC_CKAPB2_DIV4`: ADC prescaler select `CK_APB2/4`

`RCU_CKADC_CKAPB2_DIV6`: ADC prescaler select `CK_APB2/6`

`RCU_CKADC_CKAPB2_DIV8`: ADC prescaler select `CK_APB2/8`

`RCU_CKADC_CKAPB2_DIV12`: ADC prescaler select `CK_APB2/12`

`RCU_CKADC_CKAPB2_DIV16`: ADC prescaler select `CK_APB2/16`

`RCU_CKADC_CKAHB_DIV5`: ADC prescaler select `CK_AHB/5`

`RCU_CKADC_CKAHB_DIV6`: ADC prescaler select `CK_AHB/6`

RCU_CKADC_CK_AHB_DIV10: ADC prescaler select CK_AHB/10

RCU_CKADC_CK_AHB_DIV20: ADC prescaler select CK_AHB/20.

Входные аналоговые каналы, подлежащие обработке должны быть сгруппированы в регулярные (Regular) или вложенные (Inserted) группы.

В регулярную группу может входить до 16 любых каналов данного АЦП, которые группируются в необходимом порядке преобразования. Конфигурация регулярной группы осуществляется с помощью функции

```
void adc_regular_channel_config(uint32_t adc_periph , uint8_t rank , uint8_t  
adc_channel , uint32_t sample_time),
```

где:

adc_periph – номер модуля АЦП (ADCx) (x=0,1,2);

rank – номер в последовательности преобразований для данного канала (0 – 15);

adc_channel – номер аналогового канала (ADC_CHANNEL_0 - ADC_CHANNEL_9);

sample_time – время выборки (sample time) для данного канала в периодах тактового сигнала АЦП:

ADC_SAMPLETIME_1POINT5: 1.5 cycles

ADC_SAMPLETIME_7POINT5: 7.5 cycles

ADC_SAMPLETIME_13POINT5: 13.5 cycles

ADC_SAMPLETIME_28POINT5: 28.5 cycles

ADC_SAMPLETIME_41POINT5: 41.5 cycles

ADC_SAMPLETIME_55POINT5: 55.5 cycles

ADC_SAMPLETIME_71POINT5: 71.5 cycles

ADC_SAMPLETIME_239POINT5: 239.5 cycles.

Результат преобразования каждого канала, входящего в регулярную группу, записывается в один и тот же регистр ADC_RDATA. Поэтому, если в регулярную группу входит более одного канала, для чтения результата необходимо использовать канал прямого доступа к памяти. Если в регулярную группу входит один канал, то для чтения результата преобразования используется функция

```
uint16_t adc_regular_data_read(uint32_t adc_periph),
```

где adc_periph – номер модуля АЦП (ADCx) (x=0,1,2).

Во вложенную группу может входить до 4 любых каналов данного АЦП, которые группируются в необходимом порядке преобразования. Конфигурация вложенной группы осуществляется с помощью функции

```
void adc_inserted_channel_config (uint32_t adc_periph , uint8_t rank , uint8_t  
adc_channel , uint32_t sample_time),
```

где:

adc_periph – номер модуля АЦП (ADCx) (x=0,1,2);

rank – номер в последовательности преобразований для данного канала (0 – 3);

adc_channel – номер аналогового канала (ADC_CHANNEL_0 - ADC_CHANNEL_9);

sample_time – время выборки (sample time) для данного канала в периодах тактового сигнала АЦП:

ADC_SAMPLETIME_1POINT5: 1.5 cycles

ADC_SAMPLETIME_7POINT5: 7.5 cycles
 ADC_SAMPLETIME_13POINT5: 13.5 cycles
 ADC_SAMPLETIME_28POINT5: 28.5 cycles
 ADC_SAMPLETIME_41POINT5: 41.5 cycles
 ADC_SAMPLETIME_55POINT5: 55.5 cycles
 ADC_SAMPLETIME_71POINT5: 71.5 cycles
 ADC_SAMPLETIME_239POINT5: 239.5 cycles.

Результат преобразования каждого канала, входящего во вложенную группу, записывается в регистры ADC_IDATAx (x=0 - 3). Перед записью в указанные регистры к результату преобразования прибавляется смещение, задаваемое программно в соответствующих регистрах ADC_IOFFx (x=0 - 3). Для чтения результата преобразования используется функция

```
uint16_t adc_inserted_data_read(uint32_t adc_periph, uint8_t inserted_channel),
```

где:

adc_periph – номер модуля АЦП (ADCx) (x=0,1,2);

inserted_channel – номер канала в последовательности преобразований:

ADC_INSERTED_CHANNEL_0: inserted Channel0

ADC_INSERTED_CHANNEL_1: inserted channel1

ADC_INSERTED_CHANNEL_2: inserted Channel2

ADC_INSERTED_CHANNEL_3: inserted Channel3.

Количество входных аналоговых каналов в регулярной или вложенной группе задаётся функцией

```
void adc_channel_length_config(uint32_t adc_periph, uint8_t adc_channel_group, uint32_t length),
```

где:

adc_periph – номер модуля АЦП (ADCx) (x=0,1,2);

adc_channel_group – тип группы:

ADC_REGULAR_CHANNEL: regular channel group

ADC_INSERTED_CHANNEL: inserted channel group;

Length – количество каналов в группе (1 – 16 для регулярной группы, 1 – 4 для вложенной группы).

Триггер, запускающий преобразования в регулярной или вложенной группе задаётся с помощью функции

```
void adc_external_trigger_source_config(uint32_t adc_periph, uint8_t adc_channel_group, uint32_t external_trigger_source),
```

где:

adc_periph – номер модуля АЦП (ADCx) (x=0,1,2);

adc_channel_group – тип группы:

ADC_REGULAR_CHANNEL: regular channel group

ADC_INSERTED_CHANNEL: inserted channel group;

external_trigger_source – выбор триггера:

for regular channel:

ADC0_1_EXTTRIG_REGULAR_T0_CH0: timer 0 CC0 event select

ADC0_1_EXTTRIG_REGULAR_T0_CH1: timer 0 CC1 event select

ADC0_1_EXTTRIG_REGULAR_T0_CH2: timer 0 CC2 event select

ADC0_1_EXTTRIG_REGULAR_T1_CH1: timer 1 CC1 event select

ADC0_1_EXTTRIG_REGULAR_T2_TRGO: timer 2 TRGO event select
 ADC0_1_EXTTRIG_REGULAR_T3_CH3: timer 3 CC3 event select
 ADC0_1_EXTTRIG_REGULAR_T7_TRGO: timer 7 TRGO event select
 ADC0_1_EXTTRIG_REGULAR_EXTI_11 : external interrupt line 11
 ADC2_EXTTRIG_REGULAR_T2_CH0: timer 2 CC0 event select
 ADC2_EXTTRIG_REGULAR_T1_CH2: timer 1 CC2 event select
 ADC2_EXTTRIG_REGULAR_T0_CH2: timer 0 CC2 event select
 ADC2_EXTTRIG_REGULAR_T7_CH0: timer 7 CC0 event select
 ADC2_EXTTRIG_REGULAR_T7_TRGO: timer 7 TRGO event select
 ADC2_EXTTRIG_REGULAR_T4_CH0: timer 4 CC0 event select
 ADC2_EXTTRIG_REGULAR_T4_CH2: timer 4 CC2 event select
 ADC0_1_2_EXTTRIG_REGULAR_NONE: software trigger

for inserted channel:

ADC0_1_EXTTRIG_INSERTED_T0_TRGO: timer 0 TRGO event select
 ADC0_1_EXTTRIG_INSERTED_T0_CH3: timer 0 CC3 event select
 ADC0_1_EXTTRIG_INSERTED_T1_TRGO: timer 1 TRGO event select
 ADC0_1_EXTTRIG_INSERTED_T1_CH0: timer 1 CC0 event select
 ADC0_1_EXTTRIG_INSERTED_T2_CH3: timer 2 CC3 event select
 ADC0_1_EXTTRIG_INSERTED_T3_TRGO: timer 3 TRGO event select
 ADC0_1_EXTTRIG_INSERTED_EXTI_15: external interrupt line 15
 ADC0_1_EXTTRIG_INSERTED_T7_CH3: timer 7 CC3 event select
 ADC2_EXTTRIG_INSERTED_T0_TRGO: timer 0 TRGO event select
 ADC2_EXTTRIG_INSERTED_T0_CH3: timer 0 CC3 event select
 ADC2_EXTTRIG_INSERTED_T3_CH2: timer 3 CC2 event select
 ADC2_EXTTRIG_INSERTED_T7_CH1: timer 7 CC1 event select
 ADC2_EXTTRIG_INSERTED_T7_CH3: timer 7 CC3 event select
 ADC2_EXTTRIG_INSERTED_T4_TRGO: timer 4 TRGO event select
 ADC2_EXTTRIG_INSERTED_T4_CH3: timer 4 CC3 event select
 ADC0_1_2_EXTTRIG_INSERTED_NONE: software trigger

Разрешение триггера для регулярной или вложенной группы осуществляется с помощью функции

```
void adc_external_trigger_config(uint32_t adc_periph, uint8_t
adc_channel_group, ControlStatus newvalue),
```

где

adc_periph – номер модуля АЦП (ADCx) (x=0,1,2);

adc_channel_group – тип группы:

ADC_REGULAR_CHANNEL: regular channel group;

ADC_INSERTED_CHANNEL: inserted channel group;

newvalue – разрешение триггера:

ENABLE – триггер разрешён;

DISABLE - триггер запрещён.

При активизации триггера каждый модуль АЦП может работать в одном из возможных режимов. По умолчанию АЦП работает в режиме одиночного преобразования (Single Conversion). В этом режиме осуществляется преобразование аналогового канала, записанного первым в очередности каналов

для данной группы. После завершения преобразования в регулярной группе результат записывается в регистр ADC_RDATA, а во вложенной группе – в регистр ADC_IDATA0. Выставляется соответствующий запрос прерывания ADC0_1_IRQHandler или ADC2_IRQHandler.

В режиме непрерывного преобразования (Continuous Conversion) может работать только регулярная группа каналов. В этом режиме при первой активизации триггера АЦП работает аналогично режиму одиночного преобразования. Однако, после завершения первого преобразования сразу начинается следующее без ожидания очередной активизации триггера и так далее. После завершения каждого преобразования результат записывается в регистр ADC_RDATA и выставляется соответствующий запрос прерывания.

В режиме последовательных преобразований (Scan Conversion) после очередной активизации триггера запускается последовательность преобразований каналов, описанная при конфигурации данной группы каналов. Результат преобразования каждого канала, входящего в регулярную группу, записывается в один и тот же регистр ADC_RDATA. Поэтому, если в регулярную группу входит более одного канала, для чтения результата необходимо использовать канал прямого доступа к памяти. Результат преобразования каждого канала, входящего во вложенную группу, записывается в регистры ADC_IDATAx (x=0 - 3). Запрос на прерывание выставляется только после завершения всей заданной последовательности преобразований.

Режимы последовательных преобразований и непрерывного преобразования могут работать совместно. В этом случае сразу после окончания заданной последовательности преобразований группы она начинается заново.

Для включения канала прямого доступа к памяти АЦП используется функция

```
void adc_dma_mode_enable(uint32_t adc_periph),
```

где adc_periph – номер модуля АЦП (ADCx) (x=0,1,2).

В режиме автоматического вложения (Inserted Auto Insertion) последовательность преобразований вложенной группы может быть внедрена в уже начавшуюся последовательность преобразований регулярной группы по активизации соответствующего триггера.

Описанные специальные режимы работы модуля АЦП задаются с помощью функции

```
void adc_special_function_config(uint32_t adc_periph , uint32_t function ,  
ControlStatus newvalue),
```

где:

adc_periph – номер модуля АЦП (ADCx) (x=0,1,2);

function – режим работы:

ADC_SCAN_MODE: scan mode select

ADC_INSERTED_CHANNEL_AUTO: inserted channel group convert automatically

ADC_CONTINUOUS_MODE: continuous mode select.

Разрядность преобразования АЦП задаётся с помощью функции

```
void adc_resolution_config(uint32_t adc_periph , uint32_t resolution),
```

где:

adc_periph – номер модуля АЦП (ADCx) (x=0,1,2);

resolution – разрядность:

ADC_RESOLUTION_12B: 12-bit ADC resolution

ADC_RESOLUTION_10B: 10-bit ADC resolution

ADC_RESOLUTION_8B: 8-bit ADC resolution

ADC_RESOLUTION_6B: 6-bit ADC resolution.

Результат преобразования, записанный в соответствующий регистр, может быть выровнен либо относительно старшего, либо относительно младшего бита.

Результат выравнивания относительно младшего бита для регулярной и вложенной группы для разрядности 12 представлен на рис.6.5.2.

Regular group data

0	0	0	0	D11	D10	D9	D8	D7	D6	D5	D4	D3	D2	D1	D0
---	---	---	---	-----	-----	----	----	----	----	----	----	----	----	----	----

Inserted group data

Sign	Sign	Sign	Sign	D11	D10	D9	D8	D7	D6	D5	D4	D3	D2	D1	D0
------	------	------	------	-----	-----	----	----	----	----	----	----	----	----	----	----

Рис.6.5.2

Результат выравнивания относительно старшего бита для регулярной и вложенной группы для разрядности 12 представлен на рис.6.5.3.

Regular group data

D11	D10	D9	D8	D7	D6	D5	D4	D3	D2	D1	D0	0	0	0	0
-----	-----	----	----	----	----	----	----	----	----	----	----	---	---	---	---

Inserted group data

Sign	D11	D10	D9	D8	D7	D6	D5	D4	D3	D2	D1	D0	0	0	0
------	-----	-----	----	----	----	----	----	----	----	----	----	----	---	---	---

Рис.6.5.3

Синхронизация работы отдельных модулей АЦП задаётся с помощью функции

void adc_mode_config(uint32_t mode),

где mode – режим синхронизации:

ADC_MODE_FREE: all the ADCs work independently

ADC_DUAL_REGULAR_PARALLEL_INSERTED_PARALLEL: ADC0 and ADC1 work in combined regular parallel + inserted parallel mode

ADC_DUAL_REGULAR_PARALLEL_INSERTED_ROTATION: ADC0 and ADC1 work in combined regular parallel + trigger rotation mode

ADC_DUAL_INSERTED_PARALLEL_REGULAR_FOLLOWUP_FAST: ADC0 and ADC1 work in combined inserted parallel + follow-up fast mode

ADC_DUAL_INSERTED_PARALLEL_REGULAR_FOLLOWUP_SLOW: ADC0 and ADC1 work in combined inserted parallel + follow-up slow mode

ADC_DUAL_INSERTED_PARALLEL: ADC0 and ADC1 work in inserted parallel mode only

ADC_DUAL_REGULAR_PARALLEL: ADC0 and ADC1 work in regular parallel mode only

ADC_DUAL_REGULAR_FOLLOWUP_FAST: ADC0 and ADC1 work in follow-up fast mode only

ADC_DAUL_REGULAL_FOLLOWUP_SLOW: ADC0 and ADC1 work in follow-up slow mode only

ADC_DAUL_INSERTED_TRIGGER_ROTATION: ADC0 and ADC1 work in trigger rotation mode only.

Модуль АЦП имеет возможность калибровки в течение которой определяются параметры, корректирующие результаты последующих преобразований. Калибровка АЦП должна выполняться после каждого включения питания модуля и при изменении условий функционирования. Преобразования сигналов не должны запускаться и выполняться до завершения калибровки. Режим калибровки запускается с помощью функции

```
void adc_calibration_enable(uint32_t adc_periph),
```

где `adc_periph` – номер модуля АЦП (ADCx) (x=0,1,2).

Включение модуля АЦП осуществляется с помощью функции

```
void adc_enable(uint32_t adc_periph),
```

где `adc_periph` – номер модуля АЦП (ADCx) (x=0,1,2).

Запуск программного триггера преобразования осуществляется с помощью функции

```
void adc_software_trigger_enable(uint32_t adc_periph , uint8_t adc_channel_group),
```

где:

`adc_periph` – номер модуля АЦП (ADCx) (x=0,1,2);

`adc_channel_group` – тип группы каналов:

ADC_REGULAR_CHANNEL: regular channel group

ADC_INSERTED_CHANNEL: inserted channel group.

Для того, чтобы модули АЦП функционировали необходимо предварительно включить их синхронизацию. Для включения синхронизации используется функция

```
void rcu_periph_clock_enable(rcu_periph_enum periph), где periph - RCU_ADCx (x=0 - 2).
```

Пример инициализации модуля АЦП приведён ниже.

```
void ADC_config(void)
{
```

```
// Configure PC0 pin as analog ADC input
```

```
gpio_init(GPIOC, GPIO_MODE_AIN, GPIO_OSPEED_10MHZ, GPIO_PIN_0);
```

```
rcu_periph_clock_enable(RCU_ADC0);
```

```
adc_deinit(ADC0);
```

```
/* ADC mode config */
```

```
adc_mode_config(ADC_MODE_FREE);
```

```
/* ADC continuous function enable */
```

```
adc_special_function_config(ADC0,ADC_CONTINUOUS_MODE, ENABLE);
```

```
/* ADC scan function enable */
```

```
// adc_special_function_config(ADC0, ADC_SCAN_MODE, ENABLE);
```

```
/* ADC data alignment config */
```

```

adc_data_alignment_config(ADC0, ADC_DATAALIGN_RIGHT);

/* ADC channel length config */
adc_channel_length_config(ADC0, ADC_REGULAR_CHANNEL, 1);

/* ADC regular channel config */
adc_regular_channel_config(ADC0, 0, ADC_CHANNEL_10,
ADC_SAMPLETIME_13POINT5); // Sampling Time 13.5 ADC clocks. Conversion
Time 12.5 + 13.5 = 26 ADC clocks = (1/24 MHz) * 26 = 1/08 us

/* ADC trigger config */
adc_external_trigger_source_config(ADC0, ADC_REGULAR_CHANNEL,
ADC0_1_2_EXTTRIG_REGULAR_NONE);
adc_external_trigger_config(ADC0, ADC_REGULAR_CHANNEL, ENA-
BLE);

/* ADC DMA function enable */
adc_dma_mode_enable(ADC0);
/* enable ADC interface */
adc_enable(ADC0);
    for (i=0; i < 30000; i++) {}
// delay_1ms(1);
/* ADC calibration and reset calibration */
adc_calibration_enable(ADC0);

/* ADC software trigger enable */
adc_software_trigger_enable(ADC0, ADC_REGULAR_CHANNEL);
}

```

Функция void adc_deinit(uint32_t adc_periph) осуществляет начальную деинициализацию модуля АЦП.

6.6. Цифро-аналоговый преобразователь

МК GD32F303CC имеет в своём составе 2 цифро-аналоговых преобразователя (ЦАП). Разрядность каждого ЦАП может быть программно выбрана между значениями 8 или 12 бит.

Структурная схема модуля АЦП представлена на рис.6.6.1.

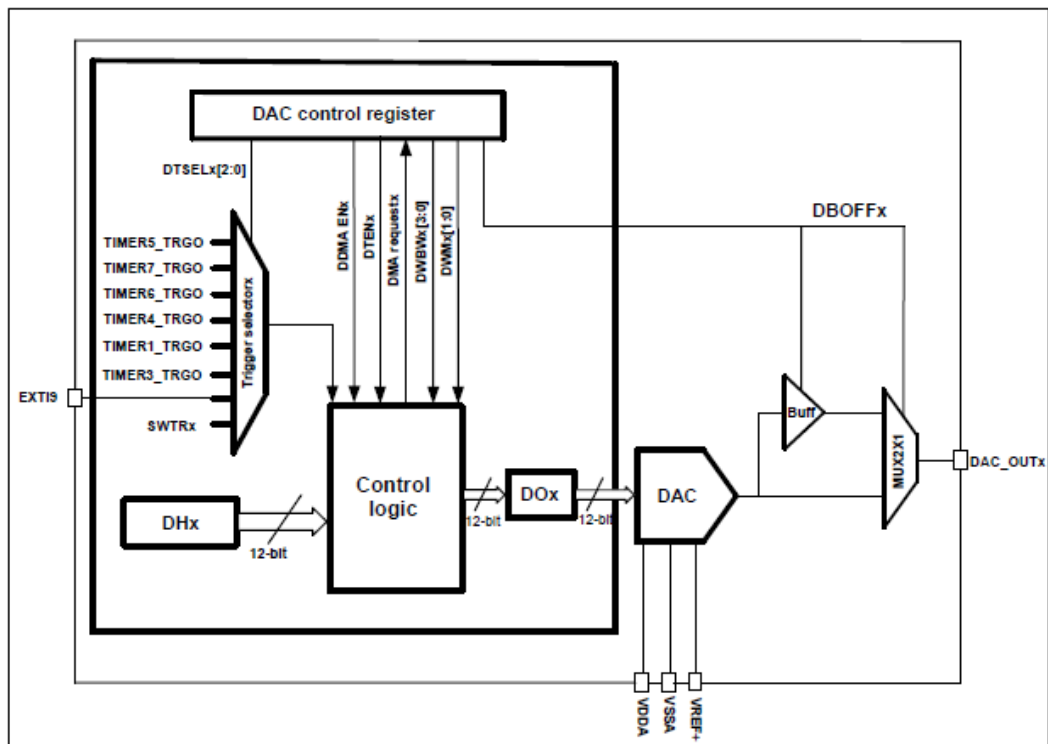


Рис.6.6.1

Модули ЦАП тактируются сигналом синхронизации периферийной шины APB1.

Преобразование в ЦАП может быть инициировано триггером или записью данных во входной регистр.

Разрешение триггера осуществляется с помощью функции `void dac_trigger_enable(uint32_t dac_periph)`,

где `dac_periph` – номер модуля ЦАП (DACx) (x=0,1).

Запрещение триггера осуществляется с помощью функции `void dac_trigger_disable(uint32_t dac_periph)`,

где `dac_periph` – номер модуля ЦАП (DACx) (x=0,1).

Выбор конкретного триггера выполняется с помощью функции `void dac_trigger_source_config(uint32_t dac_periph, uint32_t triggersource)`, где:

`dac_periph` – номер модуля ЦАП (DACx) (x=0,1);

`triggersource` – выбор конкретного триггера:

DAC_TRIGGER_T1_TRGO: TIMER1 TRGO

DAC_TRIGGER_T2_TRGO: TIMER2 TRGO

DAC_TRIGGER_T3_TRGO: TIMER3 TRGO

DAC_TRIGGER_T4_TRGO: TIMER4 TRGO

DAC_TRIGGER_T5_TRGO: TIMER5 TRGO

DAC_TRIGGER_T6_TRGO: TIMER6 TRGO
 DAC_TRIGGER_T7_TRGO: TIMER7 TRGO
 DAC_TRIGGER_EXTI_9: EXTI interrupt line9 event
 DAC_TRIGGER_SOFTWARE: software trigger

Запуск программного триггера осуществляется с помощью функции
`void dac_software_trigger_enable(uint32_t dac_periph),`

где `dac_periph` – номер модуля ЦАП (DACx) (x=0,1).

Запись данных во входной регистр ЦАП реализуется функцией

`void dac_data_set(uint32_t dac_periph, uint32_t dac_align, uint16_t data),`

где:

`dac_periph` – номер модуля ЦАП (DACx) (x=0,1);

`dac_align` – разрядность и способ выравнивания входных данных:

DAC_ALIGN_8B_R: data right 8 bit alignment

DAC_ALIGN_12B_R: data right 12 bit alignment

DAC_ALIGN_12B_L: data left 12 bit alignment;

`data` – входные данные.

Модуль ЦАП имеет подключаемый выходной буферный усилитель для обеспечения низкого выходного сопротивления. Подключение выходного буфера осуществляется функцией

`void dac_output_buffer_enable(uint32_t dac_periph),`

где `dac_periph` – номер модуля ЦАП (DACx) (x=0,1).

Отключение выходного буфера осуществляется функцией

`void dac_output_buffer_disable(uint32_t dac_periph),`

где `dac_periph` – номер модуля ЦАП (DACx) (x=0,1).

Модуль ЦАП может накладывать на основной выходной сигнал дополнительную составляющую в виде белого шума или треугольного сигнала. Белый шум реализуется с помощью аппаратного сдвигового регистра с обратной связью (рис.6.6.2).

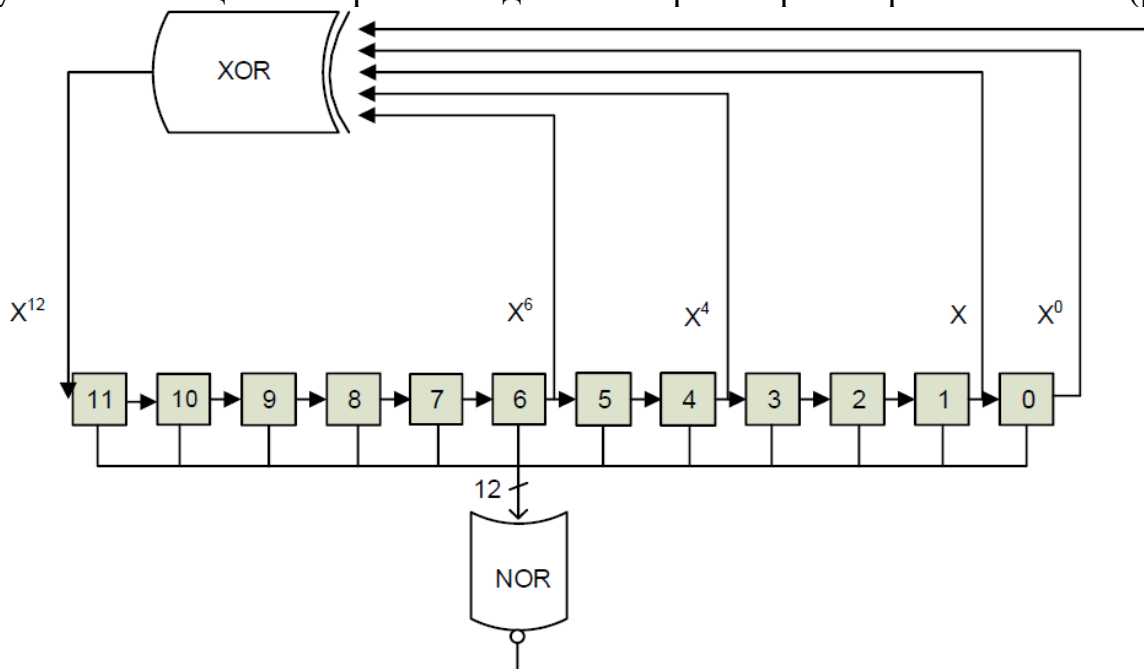


Рис.6.6.2

Выбор режима дополнительного сигнала осуществляется функцией
`void dac_wave_mode_config(uint32_t dac_periph, uint32_t wave_mode),`

где:

`dac_periph` – номер модуля ЦАП (DACx) (x=0,1);

`wave_mode` – режим дополнительного сигнала:

`DAC_WAVE_DISABLE`: нет дополнительного сигнала

`DAC_WAVE_MODE_LFSR`: белый шум

`DAC_WAVE_MODE_TRIANGLE`: треугольный сигнал.

Ширина сдвигового регистра белого шума выбирается с помощью функции

`void dac_lfsr_noise_config(uint32_t dac_periph, uint32_t unmask_bits),`

где:

`dac_periph` – номер модуля ЦАП (DACx) (x=0,1);

`unmask_bits` – немаскируемые биты сдвигового регистра:

`DAC_LFSR_BIT0`: unmask the LFSR bit0

`DAC_LFSR_BITS1_0`: unmask the LFSR bits[1:0]

`DAC_LFSR_BITS2_0`: unmask the LFSR bits[2:0]

`DAC_LFSR_BITS3_0`: unmask the LFSR bits[3:0]

`DAC_LFSR_BITS4_0`: unmask the LFSR bits[4:0]

`DAC_LFSR_BITS5_0`: unmask the LFSR bits[5:0]

`DAC_LFSR_BITS6_0`: unmask the LFSR bits[6:0]

`DAC_LFSR_BITS7_0`: unmask the LFSR bits[7:0]

`DAC_LFSR_BITS8_0`: unmask the LFSR bits[8:0]

`DAC_LFSR_BITS9_0`: unmask the LFSR bits[9:0]

`DAC_LFSR_BITS10_0`: unmask the LFSR bits[10:0]

`DAC_LFSR_BITS11_0`: unmask the LFSR bits[11:0].

Амплитуда треугольного сигнала выбирается с помощью функции

`void dac_triangle_noise_config(uint32_t dac_periph, uint32_t amplitude),`

где:

`dac_periph` – номер модуля ЦАП (DACx) (x=0,1);

`amplitude` – амплитуда треугольного сигнала:

`DAC_TRIANGLE_AMPLITUDE_1`: triangle amplitude is 1

`DAC_TRIANGLE_AMPLITUDE_3`: triangle amplitude is 3

`DAC_TRIANGLE_AMPLITUDE_7`: triangle amplitude is 7

`DAC_TRIANGLE_AMPLITUDE_15`: triangle amplitude is 15

`DAC_TRIANGLE_AMPLITUDE_31`: triangle amplitude is 31

`DAC_TRIANGLE_AMPLITUDE_63`: triangle amplitude is 63

`DAC_TRIANGLE_AMPLITUDE_127`: triangle amplitude is 127

`DAC_TRIANGLE_AMPLITUDE_255`: triangle amplitude is 255

`DAC_TRIANGLE_AMPLITUDE_511`: triangle amplitude is 511

`DAC_TRIANGLE_AMPLITUDE_1023`: triangle amplitude is 1023

`DAC_TRIANGLE_AMPLITUDE_2047`: triangle amplitude is 2047

`DAC_TRIANGLE_AMPLITUDE_4095`: triangle amplitude is 4095.

Включение модуля ЦАП осуществляется с помощью функции

`void dac_enable(uint32_t dac_periph),`

где `dac_periph` – номер модуля ЦАП (DACx) (x=0,1).

Для того, чтобы модули ЦАП функционировали необходимо предварительно включить их синхронизацию. Для включения синхронизации используется функция

```
void rcu_periph_clock_enable(rcu_periph_enum periph),  
где periph - RCU_DAC .
```

Пример инициализации модуля ЦАП приведён ниже.

```
void dac_config(void)  
{  
    gpio_init(GPIOA,          GPIO_MODE_AIN,          GPIO_OSPEED_50MHZ,  
GPIO_PIN_4);  
  
    rcu_periph_clock_enable(RCU_DAC);  
  
    dac_deinit();  
    /* configure the DAC0 */  
    //  dac_trigger_source_config(DAC0, DAC_TRIGGER_SOFTWARE);  
    //  dac_trigger_enable(DAC0);  
    dac_trigger_disable(DAC0);  
    dac_wave_mode_config(DAC0, DAC_WAVE_DISABLE);  
    //  dac_wave_mode_config(DAC0, DAC_WAVE_MODE_LFSR);  
    //  dac_lfsr_noise_config(DAC0, DAC_LFSR_BITS10_0);  
  
    dac_output_buffer_enable(DAC0);  
  
    /* enable DAC0 and set data */  
    dac_enable(DAC0);  
    //  dac_data_set(DAC0, DAC_ALIGN_12B_R, 0x7F0);  
}
```

Функция void dac_deinit(void) осуществляет начальную деинициализацию модуля ЦАП.

6.7. Система синхронизации.

Все внутренние блоки МК GD32F303CC синхронизируются несколькими тактовыми сигналами, формируемыми в системе синхронизации.

Основной тактовый сигнал, из которого формируются все остальные, это сигнал синхронизации внутренней системной шины АНВ (Advanced High Performance Bus), который называется СК_АНВ. Он может формироваться из нескольких источников, к числу которых относятся:

- внутренний высокочастотный тактовый генератор 4 – 32 МГц с синхронизацией от внешнего кварцевого резонатора;
- внутренний низкочастотный тактовый генератор 8 МГц;
- система фазовой автоподстройки частоты (Phase Locked Loop - PLL) с выходным сигналом до 120 МГц. Входным сигналом системы PLL могут быть выходные сигналы вышеперечисленных генераторов или сигнал внутреннего генератора 48 МГц.

Для данного лабораторного стенда сигнал синхронизации СК_АНВ имеет частоту 120 МГц и формируется с помощью системы PLL, тактируемой от внутреннего высокочастотного генератора с внешним кварцевым резонатором 8 МГц. Настройка сигнала синхронизации СК_АНВ осуществляется с помощью функции `void SystemInit(void)`.

Из сигнала СК_АНВ формируются тактовые сигналы двух периферийных шин APB1 (Advanced Peripheral Bus) и APB2, которые синхронизируют работу всех периферийных устройств МК. Частота сигнала APB1_СК может достигать 60 МГц, а частота сигнала APB1_СК – 120 МГц. Коэффициент деления частоты для соответствующих шин задаётся с помощью функций

```
void rcu_apb1_clock_config(uint32_t ck_apb1) и
```

```
void rcu_apb2_clock_config(uint32_t ck_apb2),
```

где `ck_apb1` – коэффициент деления частоты сигнала СК_АНВ:

```
RCU_APB1_CK_AHB_DIV1: select CK_AHB as CK_APB1
```

```
RCU_APB1_CK_AHB_DIV2: select CK_AHB/2 as CK_APB1
```

```
RCU_APB1_CK_AHB_DIV4: select CK_AHB/4 as CK_APB1
```

```
RCU_APB1_CK_AHB_DIV8: select CK_AHB/8 as CK_APB1
```

```
RCU_APB1_CK_AHB_DIV16: select CK_AHB/16 as CK_APB1;
```

`ck_apb2` – коэффициент деления частоты сигнала СК_АНВ:

```
RCU_APB2_CK_AHB_DIV1: select CK_AHB as CK_APB2
```

```
RCU_APB2_CK_AHB_DIV2: select CK_AHB/2 as CK_APB2
```

```
RCU_APB2_CK_AHB_DIV4: select CK_AHB/4 as CK_APB2
```

```
RCU_APB2_CK_AHB_DIV8: select CK_AHB/8 as CK_APB2
```

```
RCU_APB2_CK_AHB_DIV16: select CK_AHB/16 as CK_APB2
```

Если коэффициент деления частоты для соответствующей шины больше единицы, то частота сигнала синхронизации таймеров, синхронизируемых от этой шины, умножается на два.

Аналого-цифровой преобразователь может синхронизироваться от шины APB2 или от шины АНВ. Он имеет свой предварительный делитель частоты синхронизации. Его коэффициент деления задаётся с помощью функции

```
void rcu_adc_clock_config(uint32_t adc_psc),
```

где `adc_psc` - коэффициент деления частоты сигнала синхронизации АЦП:

`RCU_CKADC_CKAPB2_DIV2`: ADC prescaler select `CK_APB2/2`

`RCU_CKADC_CKAPB2_DIV4`: ADC prescaler select `CK_APB2/4`

`RCU_CKADC_CKAPB2_DIV6`: ADC prescaler select `CK_APB2/6`

`RCU_CKADC_CKAPB2_DIV8`: ADC prescaler select `CK_APB2/8`

`RCU_CKADC_CKAPB2_DIV12`: ADC prescaler select `CK_APB2/12`

`RCU_CKADC_CKAPB2_DIV16`: ADC prescaler select `CK_APB2/16`

`RCU_CKADC_CKAHB_DIV5`: ADC prescaler select `CK_AHB/5`

`RCU_CKADC_CKAHB_DIV6`: ADC prescaler select `CK_AHB/6`

`RCU_CKADC_CKAHB_DIV10`: ADC prescaler select `CK_AHB/10`

`RCU_CKADC_CKAHB_DIV20`: ADC prescaler select `CK_AHB/20`.

Включение синхронизации отдельных периферийных устройств осуществляется с помощью функции

`void rcu_periph_clock_enable(rcu_periph_enum periph),`

где `periph` – конкретное периферийное устройство (приведены в соответствующих разделах данного описания).

Пример настройки системы синхронизации приведён ниже.

`SystemInit();`

`rcu_apb1_clock_config(RCU_APB1_CKAHB_DIV4); // APB1 Clock`
`120MHz/4 = 30 MHz`

`rcu_apb2_clock_config(RCU_APB2_CKAHB_DIV4); // APB2 Clock`
`120MHz/4 = 30 MHz`

`rcu_periph_clock_enable(RCU_GPIOA); //Enable clock for`
`GPIOA.`

6.8. Система прерываний.

МК GD32F303CC поддерживают приоритетную обработку нескольких системных исключений и до 240 прерываний. Под исключением понимается нештатная ситуация, возникшая в процессе работы процессорного ядра. Обработкой сигналов запросов исключений и прерываний занимается контроллер вложенных векторных прерываний NVIC (Nested Vectored Interrupt Controller), интегрированный в процессорное ядро. На каждое прерывание или исключение процессор реагирует запуском соответствующей программы обработки прерывания (исключения). Каждому источнику прерывания присваивается конкретный приоритет и так называемый «субприоритет». Начавшаяся обработка прерываний с меньшим приоритетом может быть прервана другим исключением с большим приоритетом. Чем меньше число, обозначающее приоритет, тем больше сам приоритет данного прерывания. Самый высший возможный приоритет соответствует нулю. Субприоритет влияет на определение порядка обслуживания нескольких прерываний с равным уровнем приоритета. Прерывание с более высоким субприоритетом (обозначаемым меньшим числом) будет обслужено в первую очередь. При одинаковом уровне приоритета и субприоритета у нескольких исключений первым будет обслужено исключение с меньшим номером. Количество бит, отведённых для задания приоритета и субприоритета прерываний определяется с помощью функции

```
void nvic_priority_group_set(uint32_t nvic_prigroup),
```

где `nvic_prigroup` может быть:

`NVIC_PRIGROUP_PRE0_SUB4`: 0 bits for priority 4 bits for subpriority

`NVIC_PRIGROUP_PRE1_SUB3`: 1 bits for priority 3 bits for subpriority

`NVIC_PRIGROUP_PRE2_SUB2`: 2 bits for priority 2 bits for subpriority

`NVIC_PRIGROUP_PRE3_SUB1`: 3 bits for priority 1 bits for subpriority

`NVIC_PRIGROUP_PRE4_SUB0`: 4 bits for priority 0 bits for subpriority.

После включения питания и системного сброса обслуживание всех периферийных прерываний запрещено. Разрешение обслуживания конкретного прерывания осуществляется с помощью функции

```
void nvic_irq_enable(uint8_t nvic_irq, uint8_t nvic_irq_pre_priority,  
uint8_t nvic_irq_sub_priority),
```

где:

`nvic_irq` – номер конкретного прерывания;

`nvic_irq_pre_priority` – приоритет прерывания;

`nvic_irq_sub_priority` – субприоритет прерывания.

Запрещение обслуживания конкретного прерывания осуществляется с помощью функции

```
void nvic_irq_disable(uint8_t nvic_irq),
```

где `nvic_irq` – номер конкретного прерывания.

Глобальное запрещение обслуживания всех прерываний осуществляется с помощью макроса `__disable_irq()`, а разрешение – с помощью макроса `__enable_irq()`.

6.9. Системный таймер SYSTICK.

В процессорное ядро МК GD32F303CC включён 24-разрядный таймер, который может генерировать запросы прерывания SYSTICK. Эти запросы могут использоваться для отслеживания различных временных интервалов и моментов. Таймер синхронизируется системным тактовым сигналом процессора после деления его частоты на 8.

Инициализация таймера SYSTICK осуществляется с помощью функции

```
void systick_config(void).
```

После инициализации таймер можно использовать для реализации фиксированных временных задержек. Для этого используется функция

```
void delay_1ms(uint32_t count),
```

где count – значение временной задержки в мс.